BEGLEITHEFT ZUM ERKLÄRVIDEO · TEAM PANGMAM

Git, GitHub und GitKraken

Alles aus dem Video zum Nachblättern: wie Git funktioniert, was GitHub obendrauf legt, wie die Runde in GitKraken geht, welche Regeln bei uns gelten und was du tust, wenn etwas schiefgeht.

Verstehen	Repository · Commit · Staging · Branch · Merge · Remote
Zusammenarbeiten	Issues · Board · Pull Request · Review · CI
Klicken	GitKraken: Pull · Branch · Stage · Commit · Push
Durchhalten	Regeln · Erste Hilfe · Ticket #2

Inhalt

So benutzt du dieses Heft 5

Für wen dieses Heft ist 5
Drei Lesewege 5
Heft und Video 5
Die Kästen 6
Was dieses Heft voraussetzt 7

1 Das Ganze auf einen Blick 8

1.1 Das Problem ohne Git 8
1.2 Git, GitHub, GitKraken 9
1.3 Der Weg einer Änderung 9
1.4 Die Runde 10
1.5 Die beiden Beispiele in diesem Heft 11

TEIL I · GIT VERSTEHEN

2 Repository, Commit und die drei Bereiche 13

2.1 Das Repository: ein Ordner mit Gedächtnis 13
2.2 Der Commit: ein Schnappschuss 14
2.3 Der Steckbrief eines Commits 15
2.4 Die Kette: Eltern-Commits 15
2.5 Die drei Bereiche 16

3 Branches, HEAD, Merge und Konflikte 19

3.1 Ein Branch ist ein beweglicher Zettel 19
3.2 HEAD: Du bist hier 20
3.3 Merge: zusammenführen 21
3.4 Was ein Konflikt ist 22

4 Verteilt arbeiten mit GitHub 24

4.1 Jede Person hat das ganze Repo 24
4.2 Clone, Push und Pull 25
4.3 Fetch und Pull sind nicht dasselbe 25
4.4 Die Pfeile in GitKraken: ahead und behind 26

TEIL II · GITHUB OBENDRAUF

5 Issues, Labels und das Board 29

5.1 Ein Issue ist ein Ticket 29
5.2 Labels und Meilensteine 30
5.3 Das Board 31

6 Pull Request, Review und CI 33

6.1 Was ein Pull Request ist 33
6.2 Die Bereiche eines Pull Requests 34
6.3 Das Review 34
6.4 Die CI: automatische Prüfungen 35
6.5 Geschütztes main und Squash and merge 36

TEIL III · DIE PRAXIS IN GITKRAKEN

7 GitKraken einrichten und lesen 38

7.1 Einmal einrichten 38

7.2 Das Fenster in vier Bereichen 39
7.3 Die kleinen Zeichen 40
7.4 Die Knöpfe der Werkzeugleiste 40

8 Die Runde in zehn Schritten 42

8.1 Die Runde im Überblick 42
8.2 Schritt 1 und 2: Ticket und Branch 43
8.3 Schritt 3 und 4: Auschecken und ändern 43
8.4 Schritt 5 und 6: Committen und pushen 44
8.5 Schritt 7: Pull Request 45
8.6 Schritt 8 und 9: Review und Merge 46
8.7 Schritt 10: main pullen 46

9 Konflikte lösen 48

9.1 Weniger Konflikte von vornherein 48
9.2 Konflikt im Pull Request auf GitHub 49
9.3 Konflikt in GitKraken 49
9.4 Wann du nicht selbst auflöst 50

TEIL IV · REGELN UND ERSTE HILFE

10 Regeln und Best Practices 53

10.1 Gute Namen 53
10.2 Ablauf, Reviews und Rhythmus 54
10.3 Godot und VS Code 55
10.4 Sicherheit und Verhalten 55

11 Erste Hilfe 58

11.1 Erst die Lage, dann der Klick 58
11.2 Der Fehlerkatalog 59
11.3 Der einzige echte Notfall 60
11.4 Kurzübersicht 60

12 Deine erste Mission – Ticket #2 62

12.1 Die Aufgabe 62
12.2 Die Runde für Ticket #2 63
12.3 Closes oder Teil von? 63
12.4 Der Konflikt, der kommen wird 64
12.5 Üben ohne Risiko: das Übungs-Repo 64
12.6 Danach 64

ANHANG · NACHSCHLAGEN

A Spickzettel 67

Die Runde und die Anzeigen 67
Begriffe, Namen, Notfälle 67

B Glossar 69

A bis F 69
G bis W 70

C Lösungen zu den Selbsttests 71

Teil I: Git verstehen 71
Teil II: GitHub 72
Teil III: Die Praxis in GitKraken 72
Teil IV: Regeln und Erste Hilfe 73

D Quellen und Weiterlesen **75**
Projekt 75
Dokumentation, nach Kapiteln 75

EINSTIEG

So benutzt du dieses Heft

Das Erklärvideo zeigt dir in 23 Minuten, wie wir im Projekt Pangmam mit Git, GitHub und GitKraken arbeiten. Dieses Heft hält dasselbe schriftlich fest, etwas ausführlicher und zum Nachschlagen. Du kannst es neben dem Video lesen, danach zum Wiederholen oder später, wenn du mitten in der Arbeit hängst.

Für wen dieses Heft ist

Für alle im Team Pangmam, die schon programmieren können, aber noch nie mit Git gearbeitet haben. Du musst kein Terminal beherrschen. Wir klicken in GitKraken und arbeiten auf GitHub im Browser. Die Git-Befehle stehen trotzdem manchmal daneben, weil du sie in Fehlermeldungen, Foren und Dokumentation wiederfindest.

Am Ende kannst du eine Aufgabe vom Ticket bis zum Merge allein durchziehen: Branch anlegen, ändern, committen, pushen, Pull Request stellen, Review bekommen, mergen. Und du weißt, was du tust, wenn dabei etwas schiefgeht.

Drei Lesewege

Schnell starten
Kapitel 1 für den Überblick, dann Kapitel 8 (die Runde) und Kapitel 12 (deine erste Mission). Den Rest liest du, wenn eine Frage auftaucht.

Gründlich verstehen
Von vorn nach hinten. Teil I erklärt, was Git im Hintergrund tut. Wer das verstanden hat, klickt in GitKraken nicht mehr nach Gefühl.

Nachschlagen
Kapitel 11 ist der Fehlerkatalog, Anhang A der Spickzettel, Anhang B das Glossar. Die Zeitmarken führen dich zur passenden Stelle im Video.

Heft und Video

Jedes Kapitel nennt am Anfang die Stelle im Video, an der dasselbe Thema läuft. Das Video hat Kapitelmarken, du kannst also direkt springen.

VIDEO	AB MINUTE	HIER IM HEFT
Intro	00:00	Kapitel 1
01 Warum Versionskontrolle?	00:51	Kapitel 1
02 Git von innen	02:26	Kapitel 2
03 Branches, Merge, Konflikt	04:59	Kapitel 3
04 Verteilt arbeiten	07:08	Kapitel 4
05 GitHub obendrauf	08:32	Kapitel 5 und 6
06 GitKraken	11:41	Kapitel 7
07 Die Runde in der Praxis	12:22	Kapitel 8
08 Konflikte lösen	16:38	Kapitel 9
09 Regeln und Best Practices	17:48	Kapitel 10
10 Erste Hilfe	20:51	Kapitel 11
11 Deine erste Mission	22:10	Kapitel 12

Die Kästen

Kästen haben im ganzen Heft dieselbe Bedeutung. Wer nur die Kästen einer Sorte liest, bekommt etwas Zusammenhängendes: alle *Merke*-Kästen ergeben die Kurzfassung, alle *Achtung*-Kästen die Liste der Fallen.

> ◆ **MERKE**
> Die Kernaussage eines Abschnitts. Wenn du nur eines behältst, dann das.

> Aa **BEGRIFF**
> Ein Fachwort, in Alltagssprache erklärt. Danach benutzen wir es konsequent, damit du es in GitKraken und auf GitHub wiedererkennst.

Daneben gibt es *Achtung* für Stolperfallen, *Tipp* für Kniffe, *Im Projekt* für das, was speziell bei uns gilt, und am Kapitelende einen *Selbsttest*. Die Lösungen stehen gesammelt in Anhang C.

Was dieses Heft voraussetzt

ANNAHME	SO PRÜFST DU SIE
Du hast einen GitHub-Account mit Zwei-Faktor-Anmeldung und bist ins Repo `flxlhr/Pangmam` eingeladen.	Auf github.com/flxlhr/Pangmam siehst du oben den Reiter *Projects* und kannst eine Karte verschieben.
GitKraken ist installiert und mit GitHub verbunden. Die Oberflächen in diesem Heft entsprechen GitKraken 12.6.	In GitKraken unten rechts steht die Versionsnummer. Weichen Knöpfe ab, heißen sie meist nur leicht anders.
Die Regeln entsprechen dem Kickoff vom November 2026 und der Datei `CONTRIBUTING.md`.	Bei Widerspruch gilt `CONTRIBUTING.md` im Repo.

Außer Felix sind alle Personen in den Beispielen erfunden (Lara, Tom, Mia, Ben). Die Fenster von GitKraken und GitHub sind in diesem Heft **nachgebaut**, nicht abfotografiert. Sie zeigen nur die Bereiche, die du kennen musst, und haben nummerierte Marker. Die echten Fenster haben mehr Knöpfe. Die Originale siehst du im Video.

> • **IM PROJEKT**
>
> Fragen zu Git stellst du im Pull Request, auf Discord oder direkt beim Git-Coach. Es gibt keine dummen Git-Fragen. Es gibt nur Klicks, die man vorher hätte fragen sollen.

EINSTIEG

1 Das Ganze auf einen Blick

Fünf Leute bauen zusammen ein Spiel, und alle ändern gleichzeitig an denselben Dateien. Ohne Hilfe endet das im Chaos. Dieses Kapitel zeigt, welches Problem Git löst, wie Git, GitHub und GitKraken zusammenspielen und welchen Weg jede Änderung bei uns nimmt. Alle späteren Kapitel füllen diese Landkarte mit Einzelheiten.

DAS KANNST DU DANACH

- erklären, welches Problem Versionskontrolle löst
- Git, GitHub und GitKraken auseinanderhalten
- den Weg einer Änderung vom eigenen Rechner bis nach main beschreiben
- die zehn Stationen der Runde in der richtigen Reihenfolge nennen

HÄNGT ZUSAMMEN MIT

2 Commits · 8 Die Runde · 12 Erste Mission

Im Video: Intro ab 00:00, Kapitel 01 ab 00:51.

1.1 Das Problem ohne Git

Stell dir vor, wir legen das Spiel in einen geteilten Ordner, und alle arbeiten direkt darin. Nach einer Woche liegen dort `projekt_final`, `projekt_final_v2` und `final_v2_wirklich_final.zip`. Niemand weiß mehr, welche Datei die richtige ist.

Schlimmer: Zwei Leute öffnen dieselbe Datei, beide ändern etwas, beide speichern. Wer zuletzt speichert, gewinnt. Die Arbeit der anderen Person ist still und leise weg. Und wenn das Spiel plötzlich abstürzt, fragt sich jeder: Was hat sich seit gestern geändert, und wer war das?



Abb. 1.1 Ohne Versionskontrolle gewinnt, wer zuletzt speichert. A speichert um 10:00, B um 10:05 seine ältere Kopie darüber. Die Änderung von A ist weg, ohne dass es jemand merkt.

Ein **Versionskontrollsystem** löst genau das. Es merkt sich jeden gespeicherten Stand des Projekts, wer ihn gemacht hat und warum. Es führt die Arbeit mehrerer Leute zusammen, statt sie zu überschreiben. Und es kann jederzeit zu jedem alten Stand zurück. Das bekannteste Werkzeug dafür heißt **Git**. Fast jedes Softwareprojekt benutzt es, ab jetzt auch wir.

◆ **MERKE**

Versionskontrolle beantwortet drei Fragen zu jeder Änderung: Was hat sich geändert, wer war es, und warum? Und sie bringt Änderungen mehrerer Leute zusammen, ohne dass etwas verloren geht.

1.2 Git, GitHub, GitKraken

Drei Namen, die gern durcheinandergehen. Sie bezeichnen drei verschiedene Dinge, die in einer Kette zusammenarbeiten.

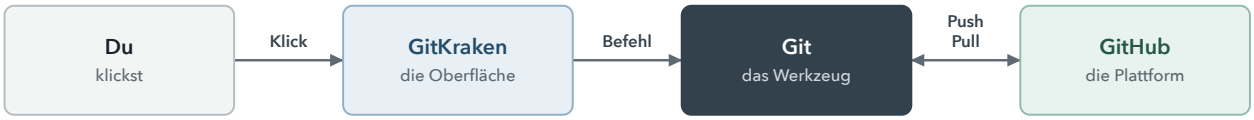


Abb. 1.2 Du klickst in GitKraken. GitKraken benutzt Git. Und Git gleicht sich mit GitHub ab.

NAME	WAS ES IST	WO ES LÄUFT
Git	Das eigentliche Werkzeug. Ein Programm, das Versionen speichert, Branches verwaltet und zusammenführt.	unsichtbar auf deinem Rechner
GitHub	Eine Webseite. Dort liegt die gemeinsame Kopie unseres Projekts, dazu Issues, das Board, Pull Requests und die automatischen Prüfungen.	im Browser auf github.com
GitKraken	Eine Oberfläche für Git. Statt Befehle ins Terminal zu tippen, klickst du.	als Programm auf deinem Rechner

▲ **GITHUB IST NICHT GIT**

Git funktioniert auch ganz ohne GitHub, nur eben allein auf deinem Rechner. GitHub ist ein Ort, an dem viele ihre Git-Repositories ablegen, und es fügt Dinge hinzu, die Git selbst nicht kennt, vor allem den Pull Request (→ Kapitel 6, S. 33).

1.3 Der Weg einer Änderung

Jede Änderung am Spiel, ob eine Zeile Code oder ein Eintrag in der Doku, nimmt denselben Weg. Sie beginnt auf deinem Rechner und endet im gemeinsamen Stand `main` auf GitHub.

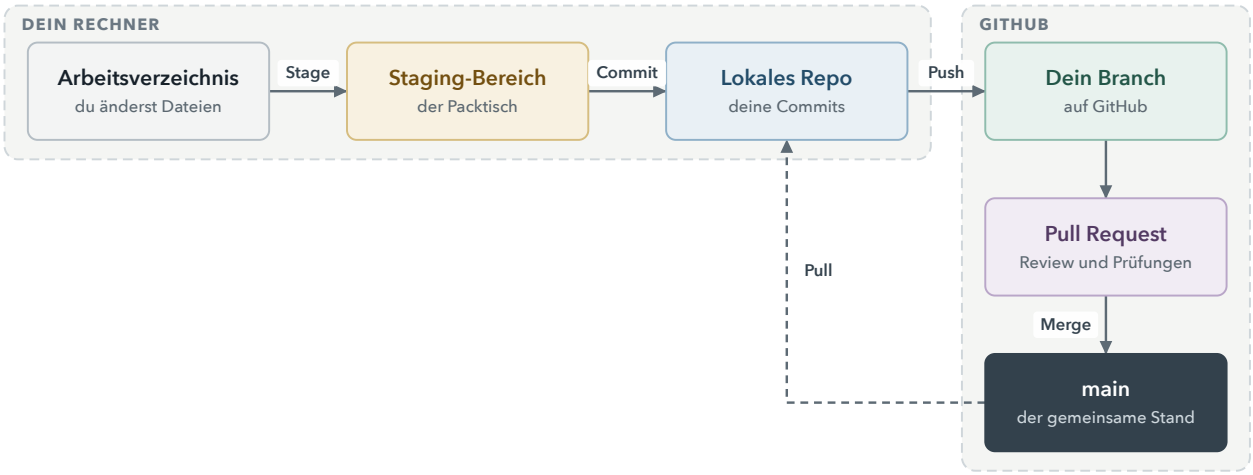


Abb. 1.3 Die Landkarte dieses Hefts. Links passiert alles auf deinem Rechner (Teil I), rechts auf GitHub (Teil II). Der gestrichelte Rückweg holt den neuen Stand wieder zu dir.

So liest du die Landkarte:

1. **Ändern.** Du bearbeitest Dateien in Godot oder VS Code. Git bemerkt die Änderung, speichert aber noch nichts (Kapitel 2).
2. **Stagen und committen.** Du legst zusammen, was zusammengehört, und speicherst es als Commit, einen dauerhaften Schnappschuss.
3. **Pushen.** Du lädst deine Commits auf GitHub hoch, auf deinen eigenen Branch (Kapitel 3 und 4).
4. **Pull Request.** Du bittest das Team, deinen Branch zu prüfen. Eine andere Person liest mit, die automatischen Prüfungen laufen (Kapitel 6).
5. **Mergen.** Ist alles grün und freigegeben, kommt deine Änderung nach `main`.
6. **Pullen.** Alle holen sich den neuen Stand. Der Kreis beginnt von vorn.

1.4 Die Runde

In der Praxis wird aus diesem Weg eine feste Abfolge von zehn Stationen. Wir nennen sie **die Runde**. Jede Aufgabe läuft genau so ab. Nach der dritten Runde machst du das im Schlaf.



Abb. 1.4 Die Runde. Grün passiert auf GitHub, blau in GitKraken, grau in deinem Editor. Nach Station 10 beginnt die nächste Aufgabe wieder bei 1.

Kapitel 8 geht jede Station mit allen Klicks durch. Kapitel 12 spielt die ganze Runde an deiner ersten echten Aufgabe durch.

1.5 Die beiden Beispiele in diesem Heft

Damit du dich nur einmal eindenken musst, kehren im ganzen Heft zwei Aufgaben aus unserem Board wieder.

Issue #10 · Bildschirm-Tastatur
Eine neue Funktion im Spiel: Ratende wählen Buchstaben per Klick auf eine Tastatur am Bildschirm. Branch `feature/10-tastatur`. An ihr erklären wir Commits, Branches und Merges.

Issue #2 · Rolle in docs/team.md eintragen
Deine erste Aufgabe: deinen Namen in die Rollentabelle schreiben. Branch zum Beispiel `docs/team-lara`. An ihr üben wir die Runde und Konflikte.

Die Datei `docs/team.md` enthält eine Tabelle mit den Rollen im Team. Jede Person trägt sich in ihrer Zeile ein. Weil alle dieselbe Datei ändern, sind Konflikte dort sicher, und das ist Absicht (→ Kapitel 12, S. 62).

docs/team.md (Ausschnitt)

```
| Rolle | Name | GitHub |
|---|---|---|
| Projektleitung & Product Owner | Felix | @flxlhr |
| Git- & Prozess-Coach | | |
| Spiel (Godot & Oberfläche) | | |
| Server (PHP, API & Betrieb) | | |
| Design (Grafik & UX) | | |
```

Zusammenfassung

- Ohne Versionskontrolle überschreiben sich Leute gegenseitig, und niemand weiß, was sich wann geändert hat.
- Git merkt sich jeden Stand mit Autor und Grund und führt die Arbeit mehrerer Leute zusammen.
- Git ist das Werkzeug, GitHub die Plattform mit der gemeinsamen Kopie, GitKraken die Oberfläche, mit der du Git per Klick bedienst.
- Jede Änderung wandert von deinem Arbeitsverzeichnis über Staging und Commit auf deinen Branch bei GitHub und per Pull Request nach `main`.
- Die Runde hat zehn Stationen: Ticket, Branch, Auschecken, Ändern, Committen, Pushen, Pull Request, Review, Merge, main pullen.

? SELBSTTEST

1. Lara und Tom ändern beide `team.md` im geteilten Cloud-Ordner. Was passiert ohne Git, und was würde Git anders machen?
2. Eine Kommilitonin sagt: „Ich hab Git nicht installiert, ich benutze doch GitHub.“ Was stimmt daran nicht?
3. Du hast in GitKraken committet. Kann das Team deine Änderung jetzt schon auf GitHub sehen? Begründe mit der Landkarte.
4. Bringe in die richtige Reihenfolge: Pull Request, Pushen, Branch, main pullen, Merge.

TEIL I

Git verstehen

Was im Hintergrund passiert, wenn du speicherst, verzweigst, zusammenführst und abgleichst.

2	**Repository, Commit und die drei Bereiche**	Was Git speichert und wie eine Änderung zum Commit wird
3	**Branches, HEAD, Merge und Konflikte**	Parallel arbeiten und wieder zusammenführen
4	**Verteilt arbeiten mit GitHub**	Clone, Push, Fetch, Pull und die Pfeile in GitKraken

TEIL I · GIT VERSTEHEN

2 Repository, Commit und die drei Bereiche

Git speichert nicht einzelne Dateien, sondern Schnappschüsse des ganzen Projekts. Jeder Schnappschuss kennt seinen Vorgänger, und so entsteht eine fälschungssichere Kette, die Geschichte des Projekts. Bevor eine Änderung in diese Kette kommt, wandert sie durch drei Bereiche. Wer diese drei Bereiche verstanden hat, versteht die rechte Hälfte von GitKraken.

DAS KANNST DU DANACH

- erklären, was ein Repository ist und wozu der Ordner `.git` da ist
- einen Commit mit seinen Bestandteilen beschreiben
- begründen, warum niemand unbemerkt einen alten Commit ändern kann
- eine Änderung über Staging gezielt in einen Commit bringen

HÄNGT ZUSAMMEN MIT

1 Überblick · 3 Branches · 7 GitKraken

Im Video: Kapitel 02 ab 02:26 (Repository 02:26, Commit 02:59, die drei Bereiche 04:07).

2.1 Das Repository: ein Ordner mit Gedächtnis

Auf deinem Rechner sieht unser Projekt aus wie ein ganz normaler Ordner: `Pangmam/` mit Unterordnern für das Spiel, den Server und die Doku. Du öffnest die Dateien in Godot oder VS Code und arbeitest damit wie immer.

Der Unterschied steckt in einem versteckten Unterordner namens `.git`. Darin hebt Git jeden Stand auf, den du je gesichert hast, dazu alle Branches und die Verbindung zu GitHub. Den Ordner samt Gedächtnis nennt man **Repository**, kurz **Repo**.



Abb. 2.1 Ein Repository besteht aus zwei Teilen. Links das Arbeitsverzeichnis mit den Dateien, die du bearbeitest. Rechts der versteckte Ordner .git, in dem die ganze Geschichte liegt.

Aa ARBEITSVERZEICHNIS (WORKING DIRECTORY)

Alles im Repo-Ordner außer `.git`: die ganz normalen Dateien, die du öffnest, änderst und speicherst. Was hier steht, ist der Stand, mit dem du gerade arbeitest.

▲ DEN ORDNER .GIT FASST DU NIE AN

Nichts darin löschen, verschieben oder von Hand bearbeiten. Das erledigt Git. Wer dort „aufräumt", beschädigt die Geschichte des Projekts. Auf dem Mac und unter Windows ist der Ordner standardmäßig ausgeblendet, und das ist gut so.

Welche Dateien Git gar nicht beachten soll, steht in der Datei `.gitignore`. Bei uns sind das zum Beispiel der Godot-Zwischenspeicher `.godot/`, der Ordner `build/` und die Datei `server/config.php` mit Passwörtern (→ Kapitel 10, S. 53).

2.2 Der Commit: ein Schnappschuss

Wenn du in einem Spiel an einen Speicherpunkt kommst, sichert das Spiel deinen ganzen Fortschritt. Läuft danach etwas schief, lädst du den Speicherpunkt und bist wieder genau dort. Ein **Commit** ist dasselbe für dein Projekt: ein gespeicherter Stand, zu dem du jederzeit zurückkannst.

Wichtig ist das Wort *ganz*. Git speichert nicht „Zeile 12 von team.md hat sich geändert", sondern einen Schnappschuss des ganzen Projekts, so wie es in diesem Moment aussieht. Trotzdem wird das Repo nicht riesig. Dateien, die sich seit dem letzten Commit nicht geändert haben, legt Git nicht noch einmal ab. Der neue Schnappschuss verweist einfach auf die alte, identische Fassung.

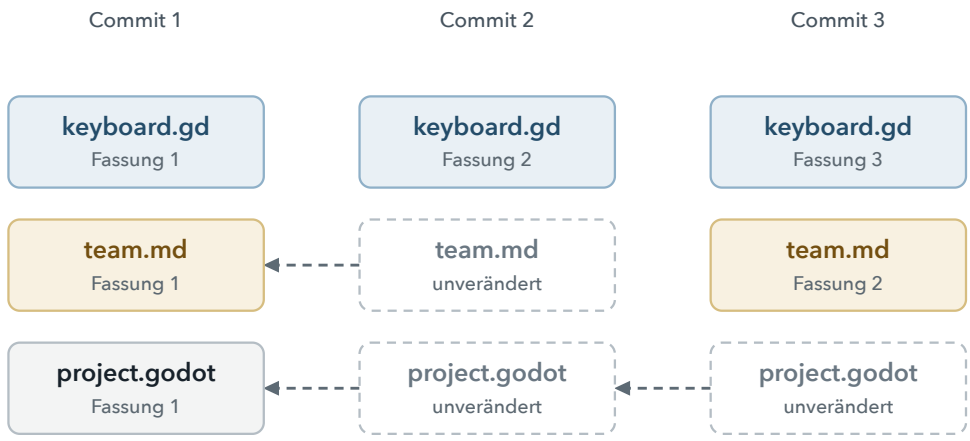


Abb. 2.2 Drei Commits, aber nur fünf gespeicherte Dateifassungen. Unveränderte Dateien (gestrichelt) verweisen auf die Fassung, die schon da ist.

◆ MERKE

Ein Commit ist ein Schnappschuss des ganzen Projekts zu einem Zeitpunkt. Zu jedem Commit kannst du jederzeit zurück.

2.3 Der Steckbrief eines Commits

Jeder Commit trägt einen Steckbrief. So sieht er aus, wenn man ihn sich mit Git anzeigen lässt:

ein Commit aus unserem Beispiel (Name erfunden)

```
commit 2dbd88179755b60d369904a98dafee62abbf1a76
Author: Lara Beispiel <lara@example.org>
Date:   Thu Nov 19 14:32:07 2026 +0100

    Tastatur färbt Treffer grün
```

TEIL	BEDEUTUNG
Hash	Die Nummer des Commits, 40 Zeichen aus 0–9 und a–f. GitKraken zeigt meist nur die ersten sechs oder sieben, hier `2dbd881`.
Autor	Name und E-Mail aus deinem GitKraken-Profil.
Datum	Wann der Commit entstanden ist.
Nachricht	Was sich ändert, in einem kurzen Satz. Regeln dafür in → Kapitel 10, S. 53.
Eltern	Der Hash des vorigen Commits. Dazu gleich mehr.
Inhalt	Der Schnappschuss selbst. GitKraken zeigt dir daraus, welche Dateien und Zeilen sich geändert haben.

Der **Hash** ist eine Art Fingerabdruck. Git berechnet ihn aus dem gesamten Inhalt des Commits, also aus Schnappschuss, Autor, Datum, Nachricht und Eltern. Ändert sich auch nur ein einziges Zeichen, entsteht ein völlig anderer Hash. Hängt man an die Nachricht oben ein Ausrufezeichen an, wird aus `2dbd881…` zum Beispiel `011a2ab…`. Ähnlichkeit gibt es nicht: Zwei Hashes verraten nichts darüber, wie ähnlich die Commits sind.

> **Aa HASH**
>
> Eine aus dem Inhalt berechnete Prüfnummer. Gleicher Inhalt ergibt immer denselben Hash, jede Änderung einen völlig anderen. Git benutzt den Hash als eindeutigen Namen jedes Commits.

2.4 Die Kette: Eltern-Commits

Jetzt der eigentliche Trick. Jeder Commit merkt sich den Hash seines Vorgängers, seines **Eltern-Commits** (parent). Dadurch hängen alle Commits in einer Kette. Diese Kette ist die Geschichte deines Projekts.

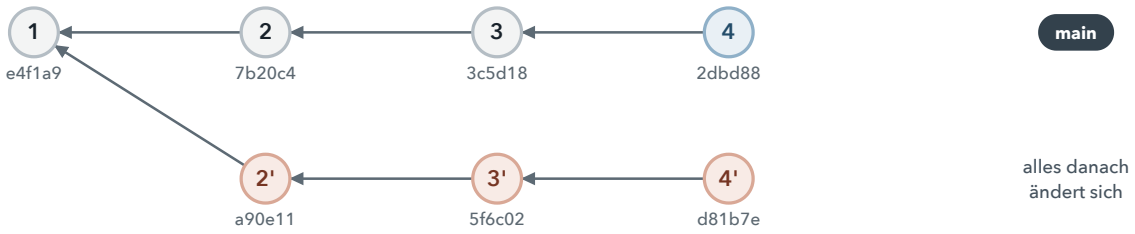


Abb. 2.3 Jeder Commit zeigt auf seinen Vorgänger. Darunter: Wer Commit 2 heimlich ändert, bekommt einen neuen Hash, und damit stimmen auch alle Hashes danach nicht mehr.

Weil der Hash des Vorgängers mit in jeden Commit eingeht, ist die Kette fälschungssicher. Wer einen alten Commit nachträglich verändert, ändert dessen Hash. Damit passt der Eltern-Eintrag im nächsten Commit nicht mehr, also ändert sich auch dessen Hash, und so weiter bis zum Ende. Jeder, der die Kette schon kennt, sieht sofort, dass etwas nicht stimmt.

> • **WAS DAS FÜR DICH HEISST**
>
> Was einmal committet und gepusht ist, bleibt in der Geschichte stehen. Das ist dein Sicherheitsnetz, denn fast nichts geht verloren (→ Kapitel 11, S. 58). Es ist aber auch der Grund, warum ein versehentlich committetes Passwort nicht durch Löschen verschwindet (→ Kapitel 10, S. 53).

2.5 Die drei Bereiche

Ein Commit entsteht nicht in einem Schritt. Deine Änderungen wandern vorher durch drei Bereiche. Genau das verwirrt am Anfang viele, deshalb in Ruhe.



Abb. 2.4 Ändern, stagen, committen. Erst der Commit nimmt die Änderung in die Geschichte auf. Mit Unstage nimmst du etwas wieder vom Packtisch.

1. **Arbeitsverzeichnis.** Du änderst Dateien und speicherst sie. Git bemerkt die Änderung und zeigt sie an, speichert aber noch nichts. In GitKraken erscheint oben im Graph die Zeile *WIP* (Work in Progress).
2. **Staging-Bereich.** Stell ihn dir als Packtisch vor. Darauf legst du genau das, was in den nächsten Commit soll. In GitKraken heißt das *Stage*, auf der Kommandozeile `git add`.
3. **Repository.** Mit *Commit* wird der Packtisch versiegelt: Git macht daraus einen Schnappschuss, gibt ihm einen Hash und hängt ihn an die Kette. Auf der Kommandozeile `git commit`.

Wozu der Zwischenschritt?

Damit jeder Commit genau eine Sache enthält. Angenommen, du arbeitest an Issue #10 und hast in `keyboard.gd` die grüne Färbung für Treffer gebaut. Nebenbei hast du in `docs/team.md` deinen Namen eingetragen. Das sind zwei verschiedene Dinge. Mit dem Packtisch machst du daraus zwei saubere Commits:



Abb. 2.5 Zwei Änderungen, zwei Commits. Jede Datei kommt einzeln auf den Packtisch und bekommt ihre eigene Nachricht.

Später sieht man in der Geschichte zwei klare Schritte statt eines Durcheinanders. Ein Reviewer kann beides getrennt lesen, und falls die Tastatur einen Fehler hat, lässt sich genau dieser eine Commit zurücknehmen.

IN GITKRAKEN	AUF DER KOMMANDOZEILE	WAS PASSIERT
Stage File / Stage All Changes	`git add <datei>` / `git add -A`	Änderung auf den Packtisch legen
Unstage File	`git restore --staged <datei>`	wieder vom Packtisch nehmen, die Änderung bleibt
Commit	`git commit -m "…"`	Packtisch als Schnappschuss versiegeln
Discard	`git restore <datei>`	Änderung im Arbeitsverzeichnis verwerfen

▲ DISCARD WIRFT WEG

Unstage nimmt nur vom Packtisch, deine Änderung bleibt. *Discard* dagegen wirft die Änderung weg. GitKraken kann nur die jeweils letzte Aktion rückgängig machen: Direkt nach einem Discard holt *Undo* ihn noch zurück. Hast du danach weitergeklickt, ist verloren, was nie committet war. Einen Papierkorb gibt es nicht (→ Kapitel 11, S. 58).

◆ MERKE

Ändern, stagen, committen. Das machst du ab jetzt ständig. Committen ist nicht Veröffentlichen: Ein Commit liegt zunächst nur auf deinem Rechner.

Zusammenfassung

- Ein Repository ist dein Projektordner plus dem versteckten Ordner `.git`, in dem die ganze Geschichte liegt. `.git` fasst du nie an.
- Ein Commit ist ein Schnappschuss des ganzen Projekts. Unveränderte Dateien speichert Git nicht doppelt.
- Jeder Commit hat einen Hash, der aus seinem Inhalt berechnet wird. Ein Zeichen anders ergibt einen völlig anderen Hash.
- Jeder Commit kennt seinen Eltern-Commit. Die Kette ist die Geschichte und kann nicht unbemerkt verändert werden.
- Änderungen wandern vom Arbeitsverzeichnis über den Staging-Bereich ins Repository: ändern, stagen, committen.
- Der Staging-Bereich erlaubt es, aus mehreren Änderungen getrennte Commits zu machen, jeder mit genau einer Sache.

? SELBSTTEST

1. Ein Teammitglied will Speicherplatz sparen und löscht den Ordner `.git`. Was ist danach noch da, und was ist weg?
2. Zwei Commits unterscheiden sich nur in einem Komma in der Nachricht. Haben sie ähnliche Hashes? Begründe.
3. Finn ändert auf seinem Rechner heimlich einen Commit von letzter Woche. Warum fällt das auf, sobald er pusht?
4. Du hast `keyboard.gd` und `team.md` geändert, willst aber nur die Tastatur committen. Was tust du in GitKraken?
5. Worin unterscheiden sich *Unstage* und *Discard*?

TEIL I · GIT VERSTEHEN

3 Branches, HEAD, Merge und Konflikte

Fünf Leute arbeiten gleichzeitig an fünf Aufgaben, und keiner soll die anderen stören. Dafür gibt es Branches, eigene Arbeitszweige. Technisch sind sie erstaunlich unspektakulär, und genau deshalb kosten sie nichts. Dieses Kapitel erklärt, was ein Branch ist, woher Git weiß, wo du gerade bist, wie Branches wieder zusammenkommen und was ein Konflikt wirklich ist.

DAS KANNST DU DANACH

- erklären, warum ein neuer Branch praktisch nichts kostet
- am Häkchen in GitKraken erkennen, auf welchem Branch du bist, und wissen, was beim Checkout passiert
- Fast-Forward und Merge-Commit unterscheiden
- einen Konflikt an seinen Markierungen erkennen und auflösen

HÄNGT ZUSAMMEN MIT

2 Commits · 4 Remote · 9 Konflikte

Im Video: Kapitel 03 ab 04:59 (Branch und HEAD 04:59, Merge 05:55, Konflikt 06:32).

3.1 Ein Branch ist ein beweglicher Zettel

Du sollst die Bildschirm-Tastatur bauen (Issue #10). Dabei wird das Spiel eine Weile halb fertig sein. Würdest du direkt im gemeinsamen Stand arbeiten, hätten alle anderen ständig dein halbes Zwischenergebnis. Deshalb arbeitest du auf einem eigenen **Branch**, einem Arbeitszweig. Der gemeinsame Hauptzweig heißt bei uns `main`.

Man könnte meinen, ein Branch sei eine Kopie des Projekts. Ist er nicht. Ein Branch ist nur ein **beweglicher Zettel, der auf einen Commit zeigt**. Technisch ist er eine winzige Datei in `.git`, in der ein einziger Hash steht:

.git/refs/heads/feature/10-tastatur

```
2dbd88179755b60d369904a98dafee62abbf1a76
```

Deshalb kostet ein neuer Branch praktisch nichts. Git kopiert dabei keine einzige Projektdatei, es schreibt nur einen neuen Zettel.

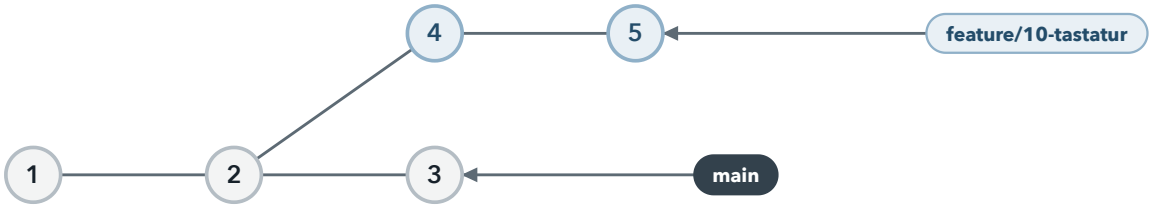


Abb. 3.1 Zwei Branches, eine gemeinsame Vergangenheit. main zeigt auf Commit 3, feature/10-tastatur auf Commit 5. Die Commits 1 und 2 gehören zu beiden.

Machst du auf deinem Branch einen neuen Commit, rutscht dein Zettel einfach mit nach vorne. Der Zettel `main` bleibt, wo er ist. So arbeiten alle nebeneinander, ohne sich zu stören.

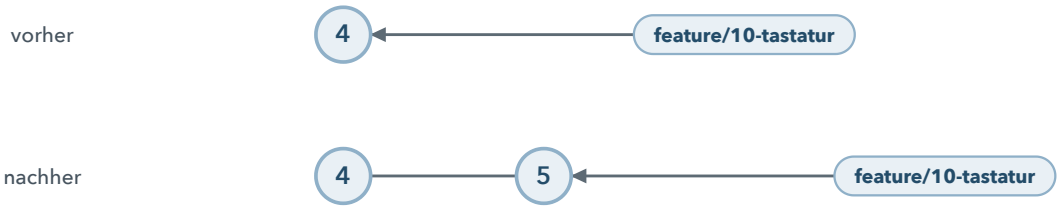


Abb. 3.2 Ein neuer Commit auf deinem Branch. Nur dein Zettel wandert mit, main bleibt stehen.

> ◆ **MERKE**
>
> Ein Branch ist ein Zettel mit einem Hash. Er zeigt auf den neuesten Commit seines Zweigs und wandert bei jedem neuen Commit mit.

3.2 HEAD: Du bist hier

Wenn es mehrere Zettel gibt, woher weiß Git dann, auf welchem Branch du gerade arbeitest und wohin dein nächster Commit kommt? Dafür gibt es einen besonderen Zeiger namens **HEAD**. HEAD zeigt auf den Branch, den du gerade ausgecheckt hast. Wie der rote Punkt auf einem Stadtplan: Du bist hier.

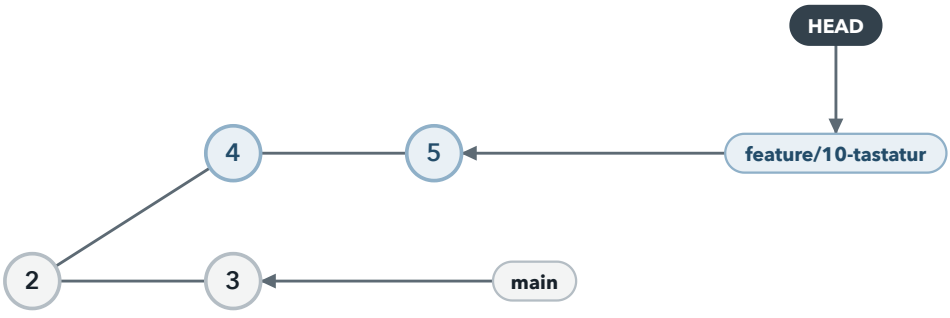


Abb. 3.3 HEAD zeigt auf deinen aktuellen Branch, und der zeigt auf einen Commit. Dein nächster Commit landet dort, wo HEAD ist.

Checkout heißt: den Branch wechseln. HEAD springt auf den anderen Zettel, und Git tauscht die Dateien in deinem Arbeitsverzeichnis gegen den Stand dieses Branches aus. Wechselst du von `feature/10-tastatur` auf `main`, verschwindet `keyboard.gd` aus deinem Ordner, weil es sie auf `main` noch nicht gibt. Sie ist nicht weg. Sie steckt in deinem Branch und taucht beim Zurückwechseln wieder auf.

In GitKraken erkennst du HEAD am **Häkchen** neben dem Branch-Namen in der linken Leiste. Ein Doppelklick auf einen anderen Branch checkt ihn aus, das Häkchen wandert mit (→ Kapitel 7, S. 38).

> **▲ VOR DEM WECHSELN: ÄNDERUNGEN SICHERN**
> Hast du ungespeicherte Änderungen (WIP) und wechselst den Branch, nimmt Git sie entweder mit oder verweigert den Wechsel, wenn sie mit dem anderen Stand kollidieren würden. Am saubersten: vorher committen, oder mit *Stash* beiseitelegen (→ Kapitel 11, S. 58).

3.3 Merge: zusammenführen

Irgendwann ist die Tastatur fertig und soll zurück nach `main`. Das heißt **Merge**, auf Deutsch zusammenführen. Bei uns passiert das immer auf GitHub über einen Pull Request (→ Kapitel 6, S. 33). Was dabei im Hintergrund geschieht, ist aber immer dasselbe, und es gibt zwei Fälle.

Fall 1: Fast-Forward

Auf `main` ist seit deinem Abzweig nichts passiert. Dann muss Git nichts zusammenbauen. Es schiebt den Zettel `main` einfach nach vorne auf deinen neuesten Commit.

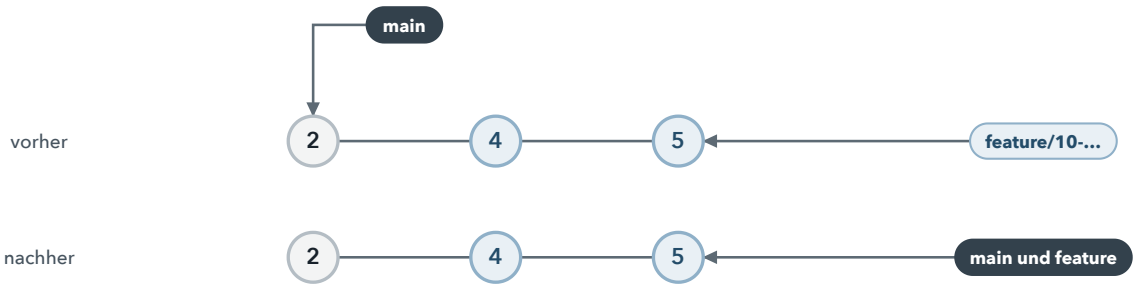


Abb. 3.4 Fast-Forward. main hatte keine eigenen neuen Commits, also rückt der Zettel einfach vor. Es entsteht kein neuer Commit.

Fall 2: Merge-Commit

Auf `main` sind inzwischen andere Commits dazugekommen, zum Beispiel die Wortanzeige von Tom. Jetzt gibt es zwei Enden. Git sucht den letzten **gemeinsamen Vorfahren** beider Zweige, vergleicht beide Enden mit ihm und baut einen neuen **Merge-Commit** mit zwei Eltern.

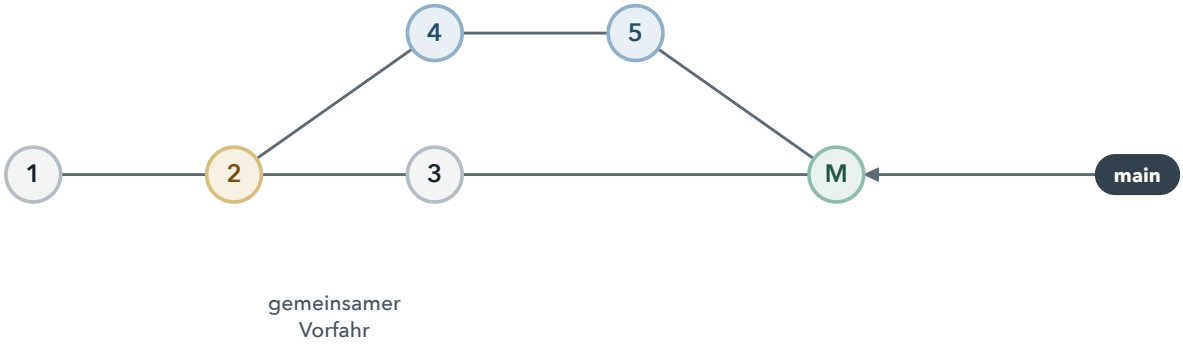


Abb. 3.5 Merge-Commit. Git vergleicht beide Enden (3 und 5) mit dem gemeinsamen Vorfahren 2 und erzeugt M mit zwei Eltern.

Meistens klappt das vollautomatisch, auch wenn beide Seiten dieselbe Datei geändert haben, solange es verschiedene Stellen sind. Git fügt dann beide Änderungen ein.

> • **BEI UNS: SQUASH AND MERGE**
>
> Auf GitHub mergen wir mit *Squash and merge*. Dabei werden alle Commits deines Branches zu einem einzigen sauberen Commit auf `main` zusammengepresst. In der Geschichte von `main` steht dann pro erledigter Aufgabe genau ein Commit (→ Kapitel 6, S. 33).

3.4 Was ein Konflikt ist

Nur in einem Fall muss Git dich fragen: wenn zwei Leute **dieselbe Zeile derselben Datei** unterschiedlich geändert haben. Dann weiß Git nicht, welche Fassung gilt. Das ist kein Fehler und nichts ist kaputt. Es ist eine Rückfrage.

Ein Beispiel aus Ticket #2: Lara und Tom tragen sich beide in Zeile 12 von `docs/team.md` als Design-Verantwortliche ein. Toms Pull Request wird zuerst gemergt. Als Lara ihren Branch mit `main` zusammenführt, hält Git an und markiert die Stelle direkt in der Datei:

docs/team.md mit Konfliktmarkierungen

```
<<<<<<< HEAD
| Design (Grafik & UX) | Lara | @lara-beispiel |
=======
| Design (Grafik & UX) | Tom | @tom-beispiel |
>>>>>>> main
```

MARKIERUNG	BEDEUTUNG
`<<<<<<< HEAD`	Ab hier deine Fassung (HEAD = wo du bist)
`=======`	Trennlinie zwischen beiden Fassungen
`>>>>>>> main`	Ende der anderen Fassung, hier die von main

So löst du ihn auf:

1. **Entscheiden.** Die eine Fassung, die andere oder eine Mischung aus beiden. Hier zum Beispiel: Wer macht Design? Kurz absprechen, dann trägt sich die andere Person in einer anderen Zeile ein.
2. **Markierungen löschen.** Alle Zeilen mit `<<<<<<<`, `=======` und `>>>>>>>` müssen raus. Übrig bleibt nur der Text, der gelten soll.
3. **Speichern und abschließen.** Die Datei speichern und den Merge mit einem Commit abschließen.

Wie das konkret auf GitHub und in GitKraken aussieht, zeigt → Kapitel 9, S. 48.

> ▲ **GODOT-SZENEN NICHT VON HAND MERGEN**
>
> `.tscn`-Dateien sind für Menschen schwer lesbar. Ein falsch aufgelöster Konflikt darin zerstört die Szene. Darum gilt: eine Szene, eine Person. Gibt es trotzdem einen Konflikt in einer `.tscn`, abbrechen und mit der anderen Person reden.

Zusammenfassung

- Ein Branch ist ein beweglicher Zettel mit einem Hash. Neue Branches kosten nichts, weil keine Dateien kopiert werden.
- Bei jedem neuen Commit wandert der Zettel des aktuellen Branches mit.
- HEAD zeigt, wo du bist. In GitKraken ist es das Häkchen. Checkout wechselt den Branch und tauscht die Dateien im Arbeitsverzeichnis aus.
- Merge führt zusammen: per Fast-Forward, wenn main nichts Neues hat, sonst mit einem Merge-Commit mit zwei Eltern.
- Ein Konflikt entsteht nur, wenn zwei Leute dieselbe Stelle unterschiedlich ändern. Du entscheidest, löschst die Markierungen und committest.

? SELBSTTEST

1. Ein Teammitglied sagt: „Ich lege lieber keinen Branch an, das verdoppelt das Projekt auf der Platte.“ Was antwortest du?
2. Du checkst `main` aus, und deine neue Datei `keyboard.gd` ist aus dem Ordner verschwunden. Ist sie verloren?
3. Seit deinem Abzweig hat niemand etwas nach main gemergt. Entsteht beim Merge ein Merge-Commit?
4. Lara ändert Zeile 12 von `team.md`, Tom Zeile 14. Gibt es einen Konflikt? Und wenn beide Zeile 12 ändern?
5. Nach dem Auflösen steht in der Datei noch eine Zeile `=======`. Was passiert, wenn du so committest?

TEIL I · GIT VERSTEHEN

4 Verteilt arbeiten mit GitHub

Bisher lief alles auf deinem Rechner. Damit fünf Leute zusammenarbeiten können, braucht es einen gemeinsamen Treffpunkt. Bei uns ist das GitHub. Dieses Kapitel erklärt, wie dein Rechner und GitHub sich abgleichen, warum Fetch und Pull nicht dasselbe sind und was die kleinen Zahlen mit Pfeilen in GitKraken bedeuten.

DAS KANNST DU DANACH

- erklären, was ein Remote ist und wofür origin steht
- Clone, Push, Fetch und Pull auseinanderhalten
- an den Anzeigen 1↑ und 1↓ in GitKraken ablesen, ob du pushen oder pullen musst
- begründen, warum man vorher pullt und nachher pusht

HÄNGT ZUSAMMEN MIT

2 Commits · 3 Branches · 8 Die Runde

Im Video: Kapitel 04 ab 07:08 (Lokal und Remote 07:08, Fetch und Pull 07:37).

4.1 Jede Person hat das ganze Repo

Git ist **verteilt**. Das heißt: Es gibt nicht einen Server, auf dem „das Projekt“ liegt und von dem alle abhängen. Jede Person hat auf ihrem Rechner eine vollständige Kopie des Repos, mit der ganzen Geschichte. Du kannst offline committen, Branches anlegen und dir alte Stände ansehen.

Damit alle zusammenkommen, gibt es zusätzlich eine gemeinsame Kopie auf GitHub. Aus Sicht deines Rechners ist sie ein **Remote**, eine entfernte Kopie. Das Standard-Remote heißt **origin**. Bei uns zeigt origin auf `github.com/flxlhr/Pangmam`.

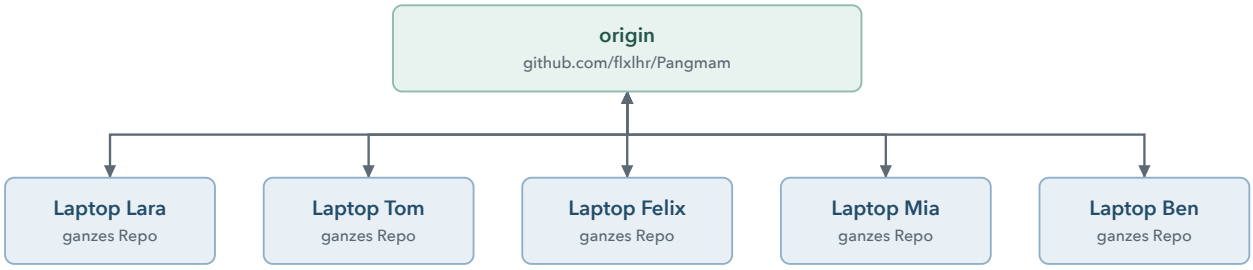


Abb. 4.1 Fünf vollständige Kopien und ein Treffpunkt. Alle gleichen sich über origin auf GitHub ab, nie direkt untereinander.

Aa REMOTE UND ORIGIN

Ein Remote ist eine Kopie des Repos an einem anderen Ort, für uns auf GitHub. *origin* ist der übliche Name für das Remote, von dem du geklont hast. In GitKraken stehen die Branches von origin links unter *Remote*.

4.2 Clone, Push und Pull

Drei Wörter decken fast alles ab, was zwischen deinem Rechner und GitHub passiert.

AKTION	RICHTUNG	WAS PASSIERT	WIE OFT
Clone	GitHub → Rechner	Das ganze Repo einmal komplett herunterladen, mit Geschichte und Verbindung zu origin	einmal ganz am Anfang
Push	Rechner → GitHub	Deine neuen Commits hochladen	nach dem Committen
Pull	GitHub → Rechner	Die neuen Commits der anderen holen und übernehmen	vor jeder neuen Arbeit

Das Zusammenspiel siehst du am besten an einem Ablauf zwischen dir, GitHub und Tom:

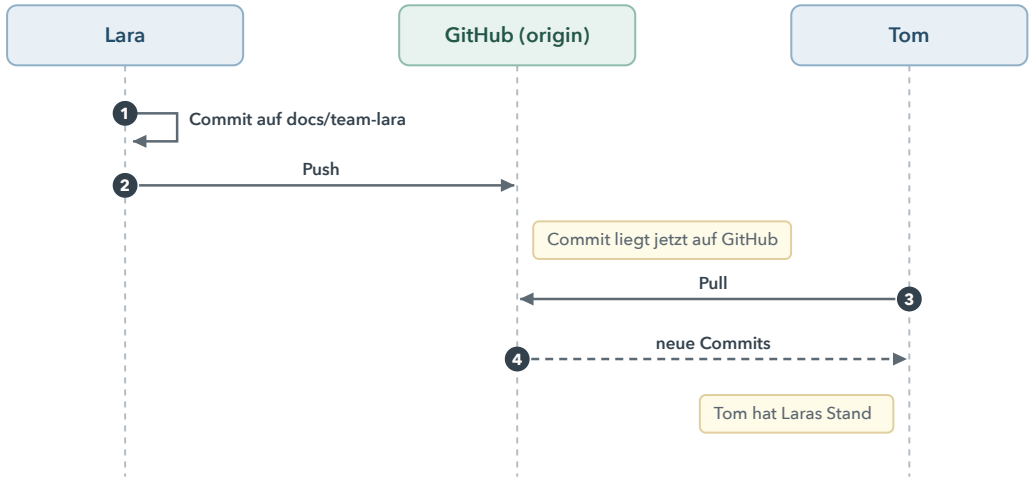


Abb. 4.2 Lara pusht ihren Commit. Tom bekommt ihn erst, wenn er selbst pullt. GitHub schickt nichts von allein auf die Rechner.

> ◆ **MERKE**
> Committen ist lokal. Erst Push bringt deine Commits zu GitHub, und erst Pull bringt die Commits der anderen zu dir. Nichts davon passiert von selbst.

4.3 Fetch und Pull sind nicht dasselbe

Git merkt sich auf deinem Rechner, wo die Branches auf GitHub zuletzt standen. Dafür hat es eigene Zettel, zum Beispiel `origin/main`. Sie heißen **Remote-Tracking-Branches**. Du arbeitest nie direkt auf ihnen. Sie sind die Notizen von Git: „Als ich zuletzt nachgeschaut habe, stand main auf GitHub hier."

Damit lässt sich der Unterschied zwischen Fetch und Pull genau sagen:

- **Fetch** lädt neue Commits von GitHub herunter und rückt nur die origin-Zettel nach. Dein eigener `main` und deine Dateien bleiben unberührt. Fetch heißt: nachschauen.
- **Pull** ist Fetch plus Merge. Erst nachschauen, dann den neuen Stand in deinen Branch übernehmen. Dein `main` rückt nach, und deine Dateien ändern sich.

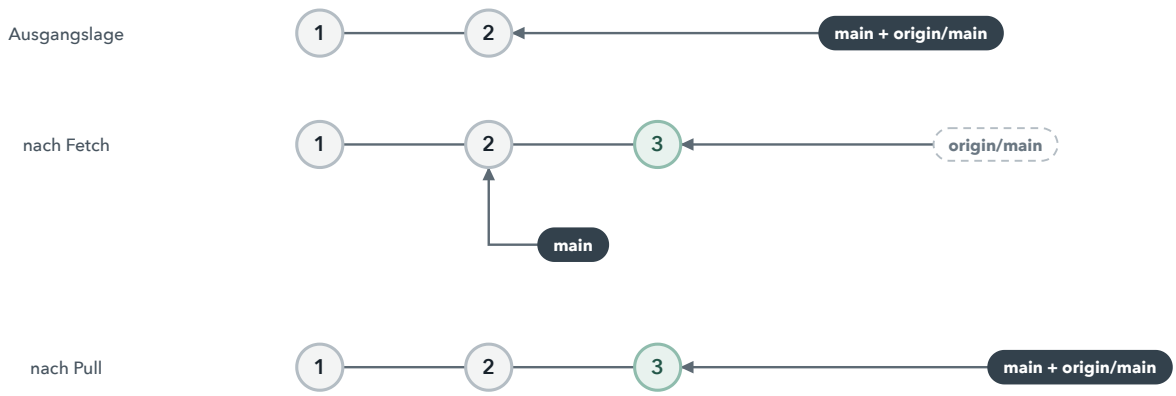


Abb. 4.3 Fetch bewegt nur den Zettel origin/main (gestrichelt). Erst Pull zieht deinen eigenen main nach.

★ GITKRAKEN HOLT VON SELBST

GitKraken macht im Hintergrund regelmäßig einen Fetch, in der Grundeinstellung jede Minute. Darum siehst du meist ohne eigenes Zutun, dass es auf GitHub etwas Neues gibt. Übernommen wird es aber erst, wenn du auf *Pull* klickst.

4.4 Die Pfeile in GitKraken: ahead und behind

Weil Git weiß, wo origin zuletzt stand, kann es den Abstand zwischen deinem Branch und GitHub zählen. GitKraken zeigt ihn neben dem Branch-Namen an.

ANZEIGE	BEDEUTUNG	ENGLISCH	WAS DU TUST
1↑	Du hast einen Commit, den GitHub noch nicht kennt.	ahead	**Push**
1↓	Auf GitHub wartet ein Commit auf dich.	behind	**Pull**
1↑ 1↓	Beides: ihr habt euch auseinanderentwickelt.	diverged	erst **Pull**, dann **Push**
keine Zahl	Dein Branch und GitHub sind gleich.	up to date	nichts

Die Zahl zählt Commits, nicht Dateien. `3↑` heißt: drei Commits liegen nur bei dir.

▲ PUSH REJECTED HEISST: ERST PULLEN

Hast du `1↑ 1↓` und pushst trotzdem, lehnt GitHub ab (*rejected*), weil dein Push die Commits der anderen überschreiben würde. Die Lösung ist immer Pull, dann Push. Nie *Force Push*, der überschreibt wirklich (→ Kapitel 11, S. 58).

◆ VORHER PULLEN, NACHHER PUSHEN

Bevor du anfängst, holst du den neuen Stand. Wenn du fertig committet hast, lädst du hoch. Dieser eine Satz verhindert die meisten Konflikte und fast jede Fehlermeldung.

Zusammenfassung

- Jede Person hat eine vollständige Kopie des Repos. GitHub ist der gemeinsame Treffpunkt, aus Sicht deines Rechners das Remote origin.
- Clone holt das Repo einmal komplett. Push lädt deine Commits hoch, Pull holt die der anderen.
- Fetch schaut nur nach und rückt die Zettel wie `origin/main` vor. Pull ist Fetch plus Merge und ändert deinen Branch.
- GitKraken zeigt den Abstand an: 1↑ heißt pushen, 1↓ heißt pullen, beides heißt erst pullen, dann pushen.
- Ein abgelehnter Push wird mit Pull und erneutem Push gelöst, nie mit Force Push.

? SELBSTTEST

1. Tom sagt: „Ich habe committet, warum sieht Lara meine Änderung nicht?“ Was fehlt?
2. Du machst Fetch. Ändern sich dadurch Dateien in deinem Arbeitsverzeichnis? Und bei Pull?
3. Neben deinem Branch steht `2↑` . Was bedeutet das, und was tust du?
4. Neben `main` steht `1↓` , und du willst einen neuen Branch anlegen. Was machst du zuerst, und warum?
5. GitHub lehnt deinen Push mit *rejected* ab. Welche zwei Schritte lösen das?

TEIL II

GitHub obendrauf

Wie wir Aufgaben verteilen und wie eine Änderung geprüft nach main kommt.

5	**Issues, Labels und das Board**	Wer macht was, und bis wann?
6	**Pull Request, Review und CI**	Wie eine Änderung geprüft nach main kommt

TEIL II · GITHUB OBENDRAUF

5 Issues, Labels und das Board

Git verwaltet Versionen, aber es sagt nicht, wer gerade woran arbeitet. Dafür nutzen wir die Aufgabenverwaltung von GitHub. Jede Aufgabe ist ein Issue, alle Issues liegen auf einem Board, und Labels und Meilensteine sortieren sie. Wer das Board lesen kann, weiß jederzeit, was als Nächstes dran ist.

DAS KANNST DU DANACH

- ein Issue lesen und erkennen, wann es fertig ist
- Labels und Meilensteine einordnen
- eine Karte auf dem Board richtig bewegen und dich als Assignee eintragen
- unterscheiden, was das Board automatisch tut und was du selbst tust

HÄNGT ZUSAMMEN MIT

6 Pull Request · 8 Die Runde · 10 Regeln

Im Video: Kapitel 05 ab 08:32 (Issues, Labels, Board).

5.1 Ein Issue ist ein Ticket

Jede Aufgabe im Projekt ist ein **Issue**, ein Ticket mit einer fortlaufenden Nummer. Mit Raute und Nummer verweist du überall darauf: in Commit-Nachrichten, in Pull Requests, in Kommentaren. Schreibst du auf GitHub `#10`, wird daraus automatisch ein Link auf Issue 10.

Ein gutes Issue hat ein **Ziel** und eine Liste **„Fertig, wenn“**. Die Liste macht aus „irgendwas mit Tastatur“ eine prüfbare Aufgabe: Sind alle Haken gesetzt, ist das Issue erledigt. So sieht unser Beispiel aus:



1. **Titel und Nummer:** worum es geht, und wie du darauf verweist.
2. **Beschreibung:** Ziel und die Liste „Fertig, wenn“.
3. **Assignees:** wer gerade daran arbeitet. Mit *assign yourself* trägst du dich ein.
4. **Labels:** Art, Priorität und Bereich.
5. **Projects:** die Spalte auf dem Board und weitere Felder.
6. **Milestone:** zu welcher Etappe das Issue gehört.
7. **Development:** hier legst du den Branch an und siehst später den Pull Request.

Abb. 5.1 Ein Issue auf GitHub, nachgebaut. Links steht, was zu tun ist, rechts, wer es tut und wohin es gehört.

★ **UNKLAR? IM ISSUE FRAGEN**

Wenn ein Issue nicht klar sagt, wann es fertig ist, frag direkt im Issue nach, bevor du anfängst. Die Antwort steht dann für alle sichtbar dort, und nicht verstreut in einem Chat.

5.2 Labels und Meilensteine

Labels sind Etiketten zum Sortieren und Filtern. Jedes Issue hat bei uns genau einen Typ, eine Priorität und meist einen Bereich.

LABEL	BEDEUTUNG
typ: feature · typ: bug	neue Funktion · Fehler
typ: design · typ: doku	Grafik, Layout · Texte, Doku
typ: aufgabe · typ: lernen	Setup, Orga · Einarbeitung
prio: muss	gehört in Version 1 (Muss-Anforderung aus dem Lastenheft)
prio: soll · prio: kann	wenn Zeit reicht · Idee für später
bereich: spiellogik · bereich: ui	Regeln · Bildschirme
bereich: server · bereich: netzwerk	PHP, Datenbank · Spiel ↔ Server
bereich: infra	CI, Deployment
good first issue	gut für den Einstieg
frage	das Team muss etwas entscheiden

★ **ISSUES FILTERN**

Unter *Issues* oben ins Suchfeld zum Beispiel `label:"good first issue"` eintippen. Dann siehst du nur Aufgaben, die sich zum Einstieg eignen.

Meilensteine sind die großen Etappen bis zur Abgabe. Jeder endet an einem Donnerstag.



Abb. 5.2 Fünf Etappen bis zur Abgabe am 28. Januar. Über Weihnachten gibt es kein Treffen, Sprint 6 läuft vom 17.12. bis 06.01.

MEILENSTEIN	WAS AM ENDE STEHEN SOLL
M0 Setup & Lernen	Alle starten das Projekt und kennen den Git-Ablauf.
M1 Lokaler Prototyp	Spielbildschirm, Tastatur, Grafik. Regeln in PHP mit Tests.
M2 Online-Prototyp	Lobby per Link, eine Runde über den Server, erste Online-Version.
M3 Version 1	Alle Muss-Anforderungen: Rollenwechsel, Punkte, ganzes Wort raten.
M4 Feinschliff & Abgabe	Extras nach Zeit, Fehler beheben, Präsentation in der letzten Übung.

5.3 Das Board

Alle Issues liegen auf unserem **Board**, einem Kanban-Board im Reiter *Projects* des Repos. Jede Karte ist ein Issue, jede Spalte ein Zustand. Eine Karte wandert von links nach rechts, bis sie fertig ist.



Abb. 5.3 Die fünf Spalten und wer die Karte bewegt. Nach *In Arbeit* (blau) schiebst du selbst, *Review* und *Fertig* setzt GitHub automatisch.

SPALTE	BEDEUTUNG	WIE DIE KARTE HINKOMMT
Backlog	gesammelt, noch nicht eingeplant	automatisch, wenn ein Issue neu angelegt wird
Bereit	klar beschrieben, kann jemand nehmen	im Donnerstagstreffen beim Planen des Sprints
In Arbeit	jemand ist dran	**du selbst**, sobald du anfängst, und du trägst dich als Assignee ein
Review	ein Pull Request wartet auf Freigabe	automatisch, sobald ein Pull Request mit dem Issue verknüpft ist. Falls nicht, schiebst du sie selbst.
Fertig	gemergt und erledigt	automatisch beim Merge

▲ IN ARBEIT UND ASSIGNEE SETZT NIEMAND FÜR DICH

Diesen einen Schritt kann GitHub nicht wissen. Trägst du dich nicht ein, nimmt womöglich eine zweite Person dieselbe Aufgabe, und ihr baut doppelt. Bei Godot-Szenen führt das sogar zu Konflikten, die sich kaum lösen lassen (→ Kapitel 10, S. 53).

Die Felder auf jeder Karte

FELD	BEDEUTUNG	BEISPIEL
Status	die Spalte auf dem Board	In Arbeit
Assignee	wer gerade daran arbeitet	dein GitHub-Name
Priorität	Muss, Soll oder Kann, wie im Lastenheft	Muss
Bereich	welcher Teil des Projekts	UI
Größe	S ein Abend · M zwei bis drei Abende · L lieber aufteilen	M
Sprint	die Woche, in der es dran ist	Sprint 2 (19.–25.11.)
Meilenstein	die größere Etappe	M1 Lokaler Prototyp

Zusammenfassung

- Jede Aufgabe ist ein Issue mit Nummer. Mit `#10` verweist du überall darauf.
- Ein gutes Issue hat ein Ziel und eine Liste „Fertig, wenn“.
- Labels sortieren nach Typ, Priorität und Bereich. Meilensteine M0 bis M4 sind die Etappen bis zum 28. Januar.
- Das Board hat fünf Spalten: Backlog, Bereit, In Arbeit, Review, Fertig.
- Neu, Review und Fertig setzt GitHub automatisch. In Arbeit und den Assignee setzt du selbst, sobald du anfängst.

? SELBSTTEST

1. Du willst Issue #12 nehmen, aber Mia steht schon als Assignee drin. Was tust du?
2. Woran erkennst du, dass Issue #10 fertig ist, ohne den Code anzusehen?
3. Du hast angefangen, aber die Karte steht noch auf *Bereit*. Welches Problem kann das auslösen?
4. Welches Label suchst du, wenn du eine einfache erste Aufgabe willst, und wie filterst du danach?

TEIL II · GITHUB OBENDRAUF

6 Pull Request, Review und CI

Der Pull Request ist der wichtigste Begriff, den GitHub zu Git hinzufügt. Er ist der Ort, an dem eine Änderung besprochen, automatisch geprüft und von einer zweiten Person freigegeben wird, bevor sie im gemeinsamen Stand landet. Zusammen mit dem geschützten main sorgt er dafür, dass niemand aus Versehen etwas für alle kaputt machen kann.

DAS KANNST DU DANACH

- erklären, was ein Pull Request ist und warum er nicht zu Git gehört
- die Bereiche eines Pull Requests finden und ein Review abgeben
- ein Issue mit `Closes #…` automatisch schließen lassen
- eine rote CI-Prüfung lesen und beheben
- begründen, warum main geschützt ist und wir mit Squash and merge mergen

HÄNGT ZUSAMMEN MIT

5 Issues und Board · 3 Branches · 8 Die Runde

Im Video: Kapitel 05 ab 09:42 (Pull Request und Review 09:42, CI und geschütztes main 10:29).

6.1 Was ein Pull Request ist

Du hast die Tastatur auf deinem Branch `feature/10-tastatur` gebaut und gepusht. Jetzt soll sie nach `main`. Direkt mergen darfst du nicht, und das ist gut so. Stattdessen stellst du eine Anfrage: „Bitte schaut euch meinen Branch an und übernehmt ihn in main.“ Diese Anfrage heißt **Pull Request**, kurz **PR**.

> ▲ **DEN PULL REQUEST GIBT ES IN GIT NICHT**
>
> Git selbst kennt nur Commits, Branches und Merges. Der Pull Request ist eine Funktion von GitHub. Andere Plattformen haben dasselbe unter anderem Namen, GitLab zum Beispiel *Merge Request*. Darum findest du den PR auch nicht im Terminal, sondern im Browser.

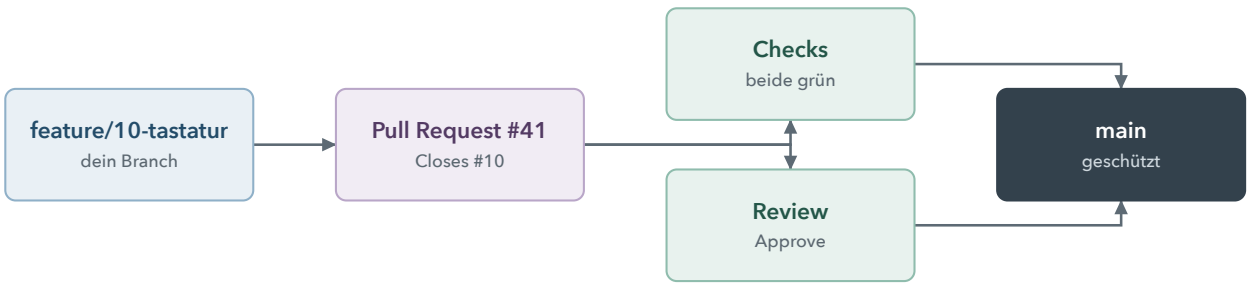
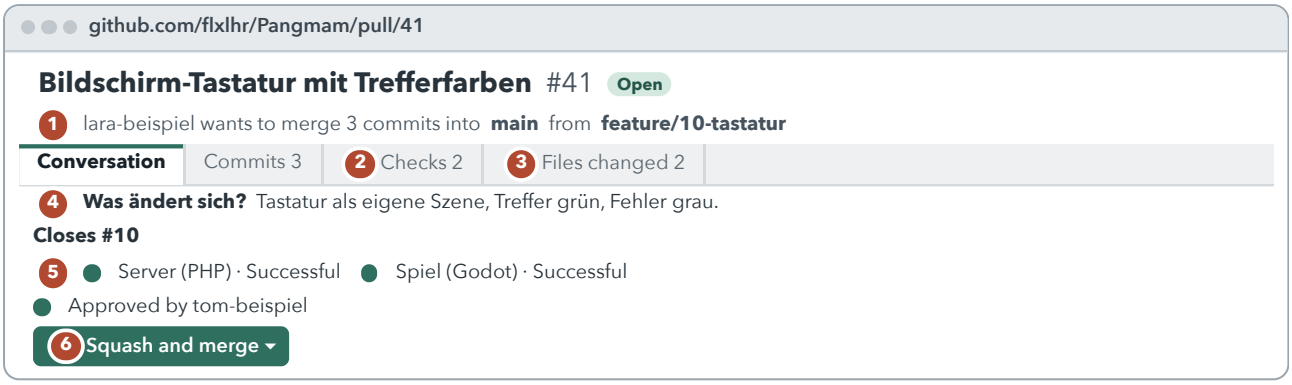


Abb. 6.1 Der Pull Request ist die Brücke von deinem Branch nach main. Auf der Brücke stehen zwei Prüfungen, die automatischen Checks und das Review. Erst wenn beide bestanden sind, öffnet sich main.

6.2 Die Bereiche eines Pull Requests

Ein Pull Request hat oben vier Reiter. Unten auf der ersten Seite steht der Kasten, in dem am Ende gemergt wird.



1. **Kopfzeile:** welcher Branch wohin soll, deiner nach main.
2. **Checks:** die Ergebnisse der automatischen Prüfungen.
3. **Files changed:** jede geänderte Zeile, alt und neu. Hier wird reviewt.
4. **Beschreibung:** was sich ändert, und `Closes #10`.
5. **Merge-Kasten:** Stand der Prüfungen und der Freigabe.
6. **Squash and merge:** wird erst aktiv, wenn alles grün und freigegeben ist.

Abb. 6.2 Ein Pull Request auf GitHub, nachgebaut. Oben die Reiter, unten der Merge-Kasten.

Closes #10: Issues automatisch schließen

Schreibst du in die Beschreibung `Closes #10`, verknüpft GitHub den PR mit Issue 10. Beim Merge nach `main` schließt sich das Issue von selbst, und die Karte springt auf *Fertig*. GitHub versteht auch `Fixes #10` und `Resolves #10`.

> ▲ **NUR DER PR, DER DIE AUFGABE ABSCHLIESST, SCHREIBT CLOSES**
>
> Erledigt ein PR nur einen Teil des Issues, schreibst du `Teil von #2`. Das ist ein normaler Verweis, der nichts schließt. Sonst wäre das Issue nach dem ersten Teil schon zu (→ Kapitel 12, S. 62).

6.3 Das Review

Beim **Review** liest eine andere Person deine Änderungen. Das Ziel ist nicht, Fehler zu suchen, um jemanden schlecht dastehen zu lassen. Das Ziel ist, dass mindestens zwei Leute jede Änderung verstehen, die in `main` landet.

1. **Files changed öffnen.** Dort steht jede geänderte Zeile. Rot ist die alte Fassung, grün die neue.
2. **An der Zeile fragen.** Über das Plus neben einer Zeile schreibst du einen Kommentar direkt an diese Stelle.
3. **Ausprobieren.** Bei Spielcode den Branch in GitKraken auschecken und das Spiel starten.
4. **Review changes abschicken.** Rechts oben mit einer der drei Entscheidungen.

ENTSCHEIDUNG	BEDEUTUNG	WANN
Comment	nur Anmerkungen, keine Entscheidung	du hast Fragen oder Hinweise, willst aber weder freigeben noch blockieren
Approve	freigeben, passt so	du hast es verstanden und siehst keinen Fehler
Request changes	bitte noch ändern	etwas ist falsch oder fehlt

• SO REVIEWEN WIR

Innerhalb von zwei Tagen, spätestens bis Mittwoch. Freundlich und konkret: „Was passiert, wenn die Lobby leer ist?“ statt „falsch“. Und gib nur frei, was du verstehst. Im Zweifel fragst du im PR nach (→ Kapitel 10, S. 53).

Bekommst du *Request changes*, ist das kein Weltuntergang. Du änderst auf demselben Branch, committest und pushst. Der PR aktualisiert sich von selbst, du musst keinen neuen stellen.

6.4 Die CI: automatische Prüfungen

CI steht für *Continuous Integration*: Jede Änderung wird sofort automatisch geprüft, bevor sie in den gemeinsamen Stand kommt. Bei uns übernimmt das **GitHub Actions**. Bei jedem Pull Request startet GitHub zwei Prüfungen:

PRÜFUNG	WAS SIE TESTET
Server (PHP)	Sind alle PHP-Dateien gültig? Laufen alle Tests durch?
Spiel (Godot)	Stimmen Code-Stil und Formatierung des GDScript-Codes? Lädt Godot das Projekt ohne Fehler?

Du findest die Ergebnisse im PR unter *Checks* und im Merge-Kasten. Ein gelber Kreis heißt: läuft noch. Ein grüner Haken: bestanden. Ein rotes Kreuz: durchgefallen.

Rot? So gehst du vor

1. **Auf Details klicken.** Neben dem roten Kreuz.
2. **Meldung lesen.** Meist steht dort Datei und Zeile, zum Beispiel `game/screens/lobby.gd:42` und was nicht stimmt.
3. **Lokal beheben, committen, pushen.** Die CI läuft dann von selbst neu.

★ FORMATIEREN VOR DEM COMMIT

Die häufigste rote Prüfung beim Spiel ist die Formatierung. `gdformat game/` im Terminal formatiert deinen GDScript-Code automatisch. Einmalig vorher installieren: `pip install gdtoolkit==4.5.0`.

◆ MERKE

Eine rote CI heißt nicht, dass du etwas kaputt gemacht hast. Sie heißt, dass die Prüfung etwas gefunden hat, bevor es in main kommt. Genau dafür ist sie da.

6.5 Geschütztes main und Squash and merge

`main` ist bei uns **geschützt**. Niemand kann direkt darauf pushen, auch nicht aus Versehen und auch nicht die Projektleitung. Jede Änderung muss durch einen Pull Request. Ein PR lässt sich erst mergen, wenn

- eine **andere Person** ihn freigegeben hat (Approve),
- **beide Prüfungen grün** sind,
- und **alle Kommentare** erledigt sind.

Das klingt streng, ist aber eine Sicherheitsleine. Auf deinem eigenen Branch kannst du nichts kaputt machen, was die anderen betrifft. Probier ruhig aus.

Gemergt wird immer mit **Squash and merge**. Dabei werden alle Commits deines Branches zu einem einzigen, sauberen Commit auf `main` zusammengepresst. Danach löscht GitHub den Branch automatisch.

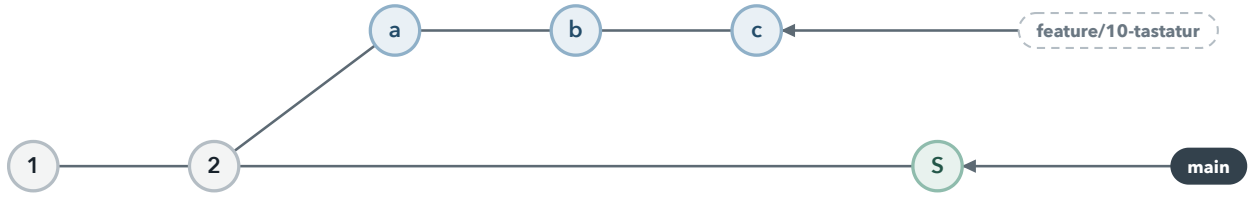


Abb. 6.3 Squash and merge. Drei Commits auf deinem Branch werden zu einem einzigen Commit S auf main. Der Branch wird danach gelöscht.

▲ NACH DEM MERGE NICHT WEITER AUF DEM ALTEN BRANCH

Weil die Commits auf main zusammengepresst wurden, passt dein alter Branch nicht mehr zur Geschichte von main. Die nächste Aufgabe bekommt immer einen frischen Branch von einem aktuellen main (→ Kapitel 8, S. 42).

Zusammenfassung

- Ein Pull Request ist eine Anfrage, einen Branch nach main zu übernehmen. Er ist eine Funktion von GitHub, nicht von Git.
- Im PR findest du Beschreibung, Commits, Checks und unter *Files changed* jede geänderte Zeile.
- `Closes #10` schließt das Issue beim Merge. Teil-PRs schreiben stattdessen `Teil von #…`.
- Beim Review entscheidet eine andere Person: Comment, Approve oder Request changes.
- Die CI prüft jeden PR automatisch. Rot heißt: Details lesen, lokal beheben, pushen.
- main ist geschützt. Gemergt wird mit Squash and merge, danach ist der Branch weg.

? SELBSTTEST

1. Jemand sucht im Terminal nach dem Befehl für einen Pull Request. Was sagst du?
2. Dein PR erledigt nur die Hälfte von Issue #10. Was schreibst du in die Beschreibung?
3. Die Prüfung *Spiel (Godot)* ist rot. Welche drei Schritte machst du?
4. Du bekommst *Request changes*. Musst du einen neuen Pull Request stellen?
5. Warum kannst du auf deinem eigenen Branch nichts kaputt machen, was die anderen betrifft?

TEIL III

Die Praxis in GitKraken

Die Oberfläche, die Runde in zehn Schritten und das Lösen von Konflikten.

7	**GitKraken einrichten und lesen**	Wo du was findest
8	**Die Runde in zehn Schritten**	Vom Ticket bis zum aktuellen main, Klick für Klick
9	**Konflikte lösen**	Auf GitHub und in GitKraken

TEIL III · DIE PRAXIS IN GITKRAKEN

7 GitKraken einrichten und lesen

GitKraken zeigt alles, was die Kapitel davor erklärt haben, in einem Fenster: Branches, Commits, den Graph, das Arbeitsverzeichnis und den Packtisch. Wer weiß, welcher Bereich was bedeutet, findet sich sofort zurecht. Dieses Kapitel richtet GitKraken einmal ein und erklärt dann die Oberfläche Bereich für Bereich.

DAS KANNST DU DANACH

- GitKraken mit GitHub verbinden und unser Repo klonen
- die vier Bereiche des Fensters benennen und ihnen die Begriffe aus Teil I zuordnen
- am Häkchen, an Laptop und Wolke und an den Pfeilen ablesen, wo ein Branch steht
- die Knöpfe der Werkzeugleiste ihrem Zweck zuordnen

HÄNGT ZUSAMMEN MIT

2 Commits · 3 Branches · 4 Remote · 8 Die Runde

Im Video: Kapitel 06 ab 11:41 (Die Oberfläche).

7.1 Einmal einrichten

Die Einrichtung dauert etwa fünf Minuten. Am einfachsten meldest du dich direkt mit deinem GitHub-Konto an, dann steht die Verbindung zu GitHub sofort.

1. **Herunterladen.** Von gitkraken.com/download, für Mac, Windows oder Linux.
2. **Anmelden.** *Sign in with GitHub* wählen und im Browser bestätigen.
3. **Verbindung prüfen.** In den Einstellungen unter **Integrations → GitHub** muss GitHub als verbunden angezeigt werden.
4. **Name und E-Mail.** Unter **Einstellungen → Profiles**. So steht dein Name an jedem deiner Commits.
5. **Repo klonen.** **Clone a repo → GitHub.com → flxlhr/Pangmam**, dann einen Ordner wählen. Das ist der Clone aus Kapitel 4.
6. **Fertig.** GitKraken öffnet das Repo und zeigt den ganzen Verlauf.

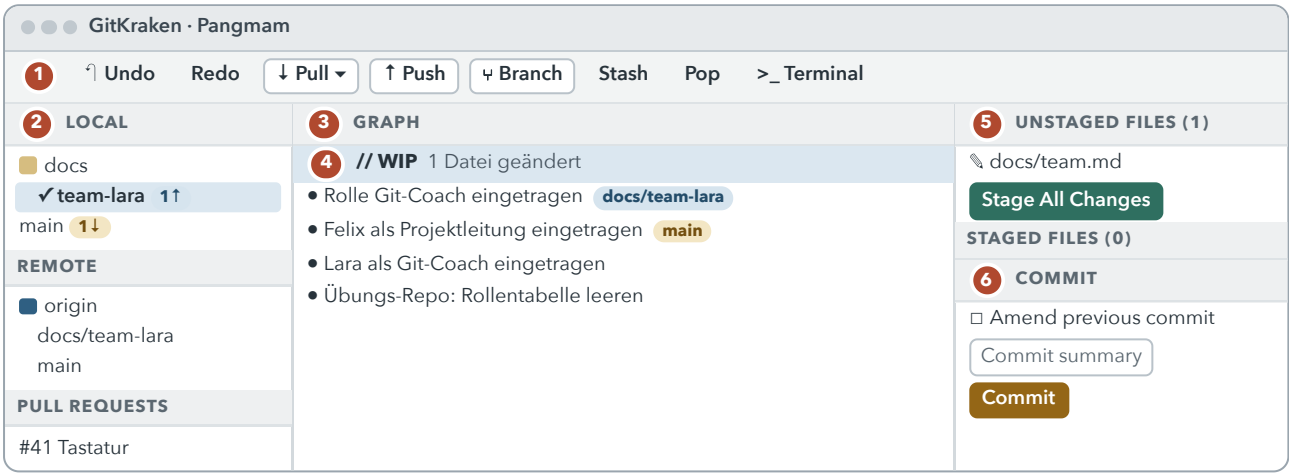
▲ NICHTS KAUFEN, KEINE ZAHLUNGSDATEN

Beim ersten Start läuft eine Testphase von GitKraken Pro. Danach geht es mit der kostenlosen Version weiter. Die reicht uns komplett, weil unser Repo öffentlich ist. Bezahlt werden müsste nur für private Repos. Mit dem GitHub Student Developer Pack gibt es Pro eine Zeit lang gratis, nötig ist es nicht.

> **▲ E-MAIL IN COMMITS IST ÖFFENTLICH**
>
> Die E-Mail aus deinem Profil steht in jedem Commit und ist in unserem öffentlichen Repo für alle lesbar. Willst du deine private Adresse nicht zeigen, nimm die *noreply*-Adresse, die GitHub dir unter **Settings → Emails** anbietet.

7.2 Das Fenster in vier Bereichen

So sieht GitKraken mit unserem Repo aus, während du an Ticket #2 arbeitest. Der Nachbau zeigt nur, was du kennen musst.



1. **Werkzeugleiste:** Pull, Push und Branch sind die wichtigsten Knöpfe, daneben Undo, Stash und Pop.
2. **Linke Leiste:** unter *Local* die Branches auf deinem Rechner, unter *Remote* die auf GitHub (origin), darunter die Pull Requests.
3. **Graph:** jeder Punkt ein Commit, jede Linie ein Branch, oben der neueste.
4. **WIP:** deine Änderungen, die noch in keinem Commit stecken.
5. **Staging-Panel:** geänderte Dateien, erst *Unstaged*, nach dem Stagen *Staged*.
6. **Commit-Bereich:** Nachricht ins Feld *Commit summary*, dann *Commit*.

Abb. 7.1 Das Hauptfenster von GitKraken, nachgebaut. Die vier Bereiche entsprechen den Begriffen aus Teil I: Branches links, die Kette in der Mitte, Arbeitsverzeichnis und Packtisch rechts.

Die rechte Leiste wechselt ihren Inhalt. Klickst du im Graph auf einen Commit, zeigt sie dessen Steckbrief: Nachricht, Autor, Hash, Eltern und die geänderten Dateien. Klickst du auf die WIP-Zeile, zeigt sie das Staging-Panel. Klickst du auf eine einzelne Datei, öffnet sich in der Mitte der Unterschied: rot die alte Zeile, grün die neue.

BEREICH IN GITKRAKEN	BEGRIFF AUS TEIL I
Local in der linken Leiste	deine Branches, die Zettel aus → Kapitel 3, S. 19
Remote → origin	die Zettel wie `origin/main` aus → Kapitel 4, S. 24
Graph	die Kette der Commits mit ihren Eltern aus → Kapitel 2, S. 13
// WIP	das Arbeitsverzeichnis mit ungespeicherten Änderungen
Unstaged Files / Staged Files	Arbeitsverzeichnis und Packtisch
Commit summary + Commit	den Packtisch als Schnappschuss versiegeln

7.3 Die kleinen Zeichen

Neben den Branch-Namen stehen kleine Zeichen. Sie beantworten drei Fragen: Wo bin ich? Wo existiert dieser Branch? Bin ich aktuell?

ZEICHEN	BEDEUTUNG
✓ Häkchen	Dieser Branch ist ausgecheckt. Das Häkchen ist dein HEAD.
Laptop-Symbol	Der Branch existiert auf deinem Rechner.
Wolke oder Avatar	Der Branch existiert auch auf GitHub (origin).
1↑	Ein Commit liegt nur bei dir. Pushen.
1↓	Ein Commit wartet auf GitHub. Pullen.
// WIP	Es gibt Änderungen, die noch in keinem Commit stecken.

Stehen Laptop und Wolke am selben Commit, sind dein Branch und GitHub gleich. Steht nur der Laptop da, hast du den Branch noch nie gepusht.

7.4 Die Knöpfe der Werkzeugleiste

KNOPF	WAS ER TUT	WANN DU IHN BRAUCHST
Pull	holt neue Commits von GitHub und übernimmt sie in den ausgecheckten Branch (Fetch plus Merge)	vor jeder neuen Arbeit, bei 1↓
Push	lädt deine Commits auf GitHub	nach dem Committen, bei 1↑
Branch	legt an der aktuellen Stelle einen neuen Branch an und checkt ihn aus	wenn du den Branch nicht auf GitHub angelegt hast
Undo / Redo	macht die letzte Aktion rückgängig, zum Beispiel einen noch nicht gepushten Commit, einen Checkout oder einen Discard. Immer nur die jeweils letzte. Auch mit Cmd / Strg + Z	sofort nach einem Klick, den du bereust
Stash	legt deine ungespeicherten Änderungen beiseite	wenn du mitten in der Arbeit den Branch wechseln musst
Pop	holt die beiseitegelegten Änderungen zurück	wenn du wieder auf dem Branch bist
Terminal	öffnet eine Kommandozeile im Repo-Ordner	für `gdformat game/`

> ★ **DOPPELKLICK HEISST CHECKOUT**
>
> Ein Doppelklick auf einen Branch in der linken Leiste checkt ihn aus. Bei einem Branch, den es nur unter *Remote* gibt, legt GitKraken dabei automatisch eine lokale Kopie an, die mit origin verbunden ist.

• DIE NAMEN KÖNNEN SICH ÄNDERN

GitKraken wird häufig aktualisiert. Die Bezeichnungen hier entsprechen Version 12.6 vom November 2026. Heißt ein Knopf bei dir leicht anders, ist es meist derselbe. Im Zweifel: fragen statt raten.

Zusammenfassung

- GitKraken richtest du einmal ein: mit GitHub anmelden, Profil prüfen, `flxlhr/Pangmam` klonen. Die kostenlose Version reicht.
- Das Fenster hat vier Bereiche: Werkzeugleiste oben, Branches links, Graph in der Mitte, Details und Staging rechts.
- Local sind deine Branches, Remote die auf GitHub. Das Häkchen zeigt, wo du bist.
- WIP sind Änderungen ohne Commit. Rechts wandern sie von Unstaged über Staged in den Commit.
- Pull, Push und Branch sind die drei wichtigsten Knöpfe. Undo, Stash und Pop retten dich aus vielen Lagen.

? SELBSTTEST

1. In welchem Bereich von GitKraken findest du den Packtisch aus Kapitel 2?
2. Neben `docs/team-lara` steht ein Laptop, aber keine Wolke. Was bedeutet das, und was fehlt noch?
3. Wie erkennst du in GitKraken, auf welchem Branch du gerade bist?
4. Du siehst oben im Graph `// WIP`. Ist deine Änderung schon auf GitHub?

TEIL III · DIE PRAXIS IN GITKRAKEN

8 Die Runde in zehn Schritten

Jede Aufgabe läuft bei uns in derselben Runde ab. Dieses Kapitel geht alle zehn Stationen durch, jeweils mit dem Ort, an dem du klickst, den genauen Knöpfen und einem Zeichen, an dem du erkennst, dass es geklappt hat. Beim ersten Mal hältst du das Heft daneben, ab der dritten Runde nicht mehr.

DAS KANNST DU DANACH

- eine Aufgabe vom Ticket bis zum Merge allein durchziehen
- zu jedem Schritt wissen, ob du auf GitHub oder in GitKraken klickst
- nach jedem Schritt prüfen, ob er geklappt hat
- die Runde mit einem aktuellen main sauber abschließen

HÄNGT ZUSAMMEN MIT

1 Überblick · 7 GitKraken · 6 Pull Request · 12 Erste Mission

---

Im Video: Kapitel 07 ab 12:22. Ticket 12:44 · Branch 12:59 · Auschecken 13:27 · Ändern 13:54 · Committen 14:18 · Pushen 14:48 · Pull Request 15:14 · Review und Merge 15:40 · main pullen 16:18.

8.1 Die Runde im Überblick

Die Runde kennst du aus → Kapitel 1, S. 8. Hier sind die zehn Stationen noch einmal, diesmal mit dem Ort, an dem du jeweils arbeitest. Als Beispiel dient Ticket #2: Lara trägt sich in `docs/team.md` als Git-Coach ein.

NR.	STATION	WO	ERGEBNIS
1	Ticket nehmen	GitHub	Du bist Assignee, Karte auf *In Arbeit*
2	Branch anlegen	GitHub	`docs/team-lara` existiert, mit dem Ticket verknüpft
3	Auschecken	GitKraken	Häkchen an `docs/team-lara`
4	Ändern	Editor	`// WIP` im Graph
5	Stagen und committen	GitKraken	neuer Commit, `1↑` am Branch
6	Pushen	GitKraken	Laptop und Wolke am selben Commit
7	Pull Request	GitHub	PR offen, Karte auf *Review*
8	Review	GitHub	Approve von einer anderen Person
9	Squash and merge	GitHub	PR *Merged*, Issue zu, Karte auf *Fertig*
10	main pullen	GitKraken	`main` ohne `1↓`, alter Branch gelöscht

8.2 Schritt 1 und 2: Ticket und Branch

1 · Das Ticket nehmen

1. **Board öffnen.** Im Repo oben auf *Projects*, dann unser Board. Ein Issue aus der Spalte *Bereit* aussuchen und öffnen.
2. **Dich eintragen.** Rechts unter *Assignees* auf *assign yourself* klicken. Jetzt steht dein Name dran.
3. **Status setzen.** Rechts unter *Projects* den Status auf *In Arbeit* stellen.

Geklappt, wenn: Auf dem Board steht die Karte in *In Arbeit* und zeigt deinen Avatar.

2 · Den Branch anlegen

Am bequemsten legst du den Branch direkt im Issue an. Dann ist er automatisch mit dem Ticket verknüpft.

1. **Create a branch.** Im Issue rechts unter *Development* auf *Create a branch* klicken.
2. **Namen kürzen.** GitHub schlägt einen sehr langen Namen aus dem Issue-Titel vor. Ersetze ihn nach unserem Schema, hier `docs/team-lara`, bei Code-Aufgaben zum Beispiel `feature/10-tastatur` (Kapitel 10).
3. **Von main starten.** Als Quelle (*Branch source*) muss `main` eingestellt sein. Dann *Create branch*.

Geklappt, wenn: Unter *Development* im Issue steht dein Branch.

> ▲ **NIE TEST, NIE FINAL2**
>
> Ein Branch-Name sagt, wozu der Branch gehört. `test`, `mein-branch` oder `final2` weiß in drei Wochen niemand mehr einzuordnen (→ Kapitel 10, S. 53).

8.3 Schritt 3 und 4: Auschecken und ändern

3 · Auschecken

Dein Branch liegt jetzt auf GitHub, aber noch nicht auf deinem Rechner. GitKraken muss ihn erst sehen.

1. **Pull.** In GitKraken `main` doppelklicken. Zeigt `main` eine `1↓`, oben auf *Pull* klicken.
2. **Branch finden.** Links unter **Remote → origin** taucht dein neuer Branch auf.
3. **Doppelklick.** GitKraken legt eine lokale Kopie an und checkt sie aus.

Geklappt, wenn: Unter *Local* steht `docs/team-lara` mit Häkchen.

> ★ **BRANCH DIREKT IN GITKRAKEN ANLEGEN**
>
> Geht auch: `main` auschecken, *Pull*, dann oben auf *Branch*, Namen eingeben, Enter. Der Branch ist dann aber erst nach dem ersten Push auf GitHub und nicht automatisch mit dem Issue verknüpft. Das holst du im Pull Request mit `Closes #2` nach.

4 · Ändern

1. **Öffnen.** Den ganzen Repo-Ordner in VS Code öffnen, oder das Spiel in Godot über `game/project.godot`.
2. **Ändern und speichern.** Hier: in `docs/team.md` in der Zeile *Git- & Prozess-Coach* deinen Namen und GitHub-Namen eintragen.

Geklappt, wenn: In GitKraken erscheint oben im Graph `// WIP`. VS Code markiert die Datei mit **M** (modified, geändert). Eine neu angelegte Datei hätte ein **U** (untracked, Git kennt sie noch nicht).

Klickst du in GitKraken auf die Datei, siehst du den Unterschied:

Unterschied in docs/team.md

```
- | Git- & Prozess-Coach | | |
+ | Git- & Prozess-Coach | Lara | @lara-beispiel |
```

Rot mit Minus ist die alte Zeile, grün mit Plus die neue. Git denkt in ganzen Zeilen: Eine geänderte Zeile erscheint als „alte raus, neue rein“.

8.4 Schritt 5 und 6: Committen und pushen

5 · Stagen und committen

1. **Stagen.** Auf die Zeile `// WIP` klicken. Rechts stehen deine Dateien unter *Unstaged Files*. Mit *Stage All Changes* legst du sie auf den Packtisch, mit *Stage File* einzelne.
2. **Hinschauen.** Unter *Staged Files* dürfen nur Dateien stehen, die zu deiner Aufgabe gehören.
3. **Nachricht.** Ins Feld *Commit summary* einen kurzen deutschen Satz, was sich ändert: `Lara als Git-Coach eingetragen`.
4. **Commit.** Auf den Commit-Knopf klicken.

Geklappt, wenn: Oben im Graph steht dein neuer Commit, und neben deinem Branch eine `1↑`. Der Commit liegt erst auf deinem Rechner.

> ▲ **NUR DEINE DATEIEN STAGEN**
>
> Godot ändert beim Öffnen manchmal nebenbei Dateien, die nichts mit deiner Aufgabe zu tun haben, zum Beispiel `project.godot` oder Szenen, die du nur angesehen hast. Solche Dateien nicht mit committen. Rechtsklick auf die Datei und *Discard changes* verwirft die ungewollte Änderung. Bei Unsicherheit: fragen.

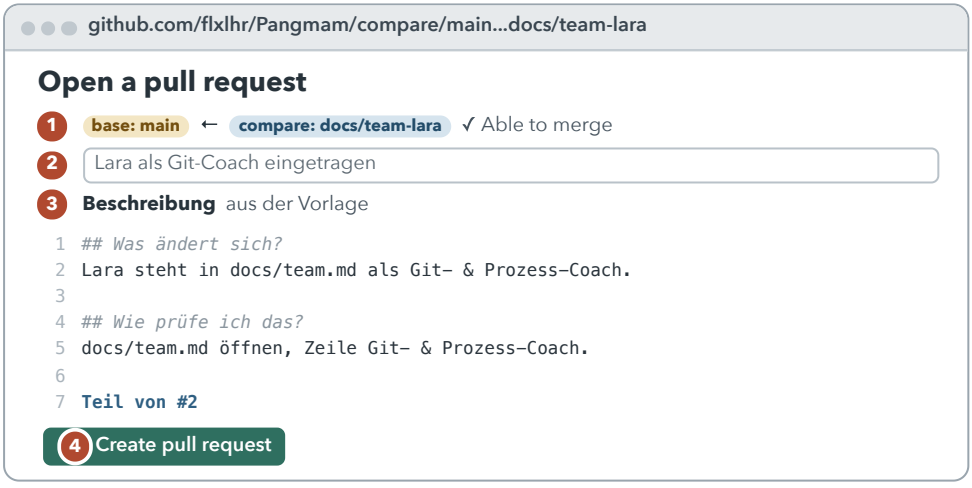
6 · Pushen

1. **Push.** Oben auf *Push* klicken.
2. **Beim ersten Mal bestätigen.** Ist der Branch auf GitHub noch unbekannt, fragt GitKraken, unter welchem Namen er dort angelegt werden soll. Vorschlag übernehmen, bestätigen.

Geklappt, wenn: Die `1↑` ist weg, und am Branch stehen Laptop und Wolke am selben Commit: Er existiert jetzt bei dir und auf GitHub.

8.5 Schritt 7: Pull Request

1. **Compare & pull request.** GitHub bemerkt deinen Push und zeigt auf der Repo-Seite einen gelben Hinweis mit diesem Knopf.
2. **Richtung prüfen.** Oben steht *base: main ← compare: docs/team-lara*. Deiner soll nach main.
3. **Titel.** Ein klarer Titel, zum Beispiel *Lara als Git-Coach eingetragen*.
4. **Vorlage ausfüllen.** Was ändert sich, wie kann man es prüfen. Dazu die Zeile `Closes #2`, wenn dein PR das Ticket abschließt, sonst `Teil von #2`.
5. **Create pull request.**



1. **base ← compare:** wohin (main) und woher (dein Branch).
2. **Titel:** ein klarer Satz, wie eine gute Commit-Nachricht.
3. **Beschreibung:** die Vorlage ausfüllen, dazu `Teil von #2` oder `Closes #2`.
4. **Create pull request:** stellt den PR. Ab jetzt laufen die Prüfungen.

Abb. 8.1 Das Formular für einen neuen Pull Request, nachgebaut. Die Richtung steht oben, die Verknüpfung mit dem Ticket in der Beschreibung.

Geklappt, wenn: Der PR ist offen, die Prüfungen laufen (gelbe Kreise), und auf dem Board springt die Karte auf *Review*.

8.6 Schritt 8 und 9: Review und Merge

8 · Review

Jetzt ist eine andere Person dran. Sie öffnet *Files changed*, kommentiert über das Plus neben einer Zeile und entscheidet mit *Review changes*: *Comment*, *Approve* oder *Request changes* (→ Kapitel 6, S. 33). Bekommst du *Request changes*, änderst du auf demselben Branch, committest und pushst. Der PR aktualisiert sich von selbst.

> • **DU BIST AUCH REVIEWER**
>
> Reviews sind keine Aufgabe für Experten. Gerade wer neu ist, merkt, ob eine Änderung verständlich ist. Reviews machen wir innerhalb von zwei Tagen, spätestens bis Mittwoch.

9 · Squash and merge

1. **Warten, bis alles grün ist.** Beide Prüfungen bestanden, eine Freigabe von jemand anderem, alle Kommentare erledigt. Dann wird der Knopf aktiv.
2. **Squash and merge.** Klicken, die vorgeschlagene Commit-Nachricht prüfen, mit *Confirm squash and merge* bestätigen.

Geklappt, wenn: Der PR zeigt *Merged*, der Branch auf GitHub ist gelöscht, Issue #2 ist geschlossen (falls `Closes #2` drinstand), und die Karte steht auf *Fertig*.

8.7 Schritt 10: main pullen

Den Schritt vergessen viele. Dein `main` auf dem Rechner weiß vom Merge noch nichts. Er steht noch auf dem alten Stand.

1. **main auschecken.** Links `main` doppelklicken. Er zeigt eine `1↓`: der gemergte Commit.
2. **Pull.** Oben auf *Pull*. Jetzt bist du aktuell.
3. **Alten Branch löschen.** Rechtsklick auf `docs/team-lara` unter *Local*, dann *Delete*. Er hat seinen Job erledigt.

Geklappt, wenn: `main` hat Häkchen und keine Pfeile mehr, und dein alter Branch ist weg. Die nächste Runde beginnt bei Schritt 1, mit einem frischen Branch von diesem aktuellen `main`.

> ◆ **MERKE**
>
> Eine Runde endet nicht mit dem Merge, sondern mit einem aktuellen main auf deinem Rechner. Wer das vergisst, startet die nächste Aufgabe auf einem alten Stand und bekommt unnötige Konflikte.

Zusammenfassung

- Ticket und Branch entstehen auf GitHub: dich eintragen, Status *In Arbeit*, unter *Development* einen Branch mit unserem Namensschema anlegen.
- In GitKraken pullst du, checkst den Branch per Doppelklick aus und erkennst am Häkchen, dass du dort bist.
- Nach dem Ändern stagst du nur deine Dateien, committest mit einer klaren Nachricht und pushst. Laptop und Wolke am selben Commit zeigen, dass alles oben ist.
- Der Pull Request bekommt einen klaren Titel und `Closes #…` oder `Teil von #…`.
- Gemergt wird nach Freigabe und grünen Prüfungen mit Squash and merge.
- Zum Schluss main pullen und den alten Branch löschen.

? SELBSTTEST

1. Du hast den Branch auf GitHub angelegt, siehst ihn aber in GitKraken nicht. Was fehlt?
2. Unter *Staged Files* steht neben `docs/team.md` auch `game/project.godot`, das du nie bewusst geändert hast. Was tust du?
3. Nach dem Push steht am Branch nur der Laptop. Was ist schiefgegangen?
4. Der PR ist gemergt. Welche zwei Dinge machst du danach noch in GitKraken?
5. Warum fängst du die nächste Aufgabe nicht einfach auf dem alten Branch an?

TEIL III · DIE PRAXIS IN GITKRAKEN

9 Konflikte lösen

Ein Konflikt ist eine Rückfrage von Git: Zwei Leute haben dieselbe Stelle geändert, welche Fassung gilt? Was technisch dahintersteckt, erklärt Kapitel 3. Hier geht es um die Praxis. Du lernst, wie du Konflikte vermeidest, wie du sie im Pull Request auf GitHub löst und wie in GitKraken. Und wann du besser abbrichst.

DAS KANNST DU DANACH

- mit einem aktuellen Branch unnötige Konflikte vermeiden
- einen Konflikt in einer Textdatei direkt auf GitHub lösen
- den Merge-Editor von GitKraken bedienen und einen Merge abbrechen
- entscheiden, wann du nicht selbst auflöst, sondern mit der anderen Person redest

HÄNGT ZUSAMMEN MIT

3 Branches · 8 Die Runde · 11 Erste Hilfe

Im Video: Kapitel 08 ab 16:38 (Konflikt auf GitHub 16:38, Konflikt in GitKraken 17:09).

9.1 Weniger Konflikte von vornherein

Die meisten Konflikte entstehen, weil jemand lange auf einem alten Stand arbeitet. Drei Gewohnheiten halten sie klein:

1. **Vor jedem neuen Branch main pullen.** Dann startest du auf dem neuesten Stand.
2. **Kleine Pull Requests.** Je kürzer ein Branch lebt, desto weniger kann sich auf main in der Zwischenzeit ändern.
3. **Zwischendurch aktualisieren.** Wurde auf main etwas gemergt, während du noch arbeitest: in GitKraken Rechtsklick auf `main`, dann *Merge main into docs/team-lara*. So holst du die Neuerungen in deinen Branch und löst einen eventuellen Konflikt gleich bei dir, statt später im PR.

> ◆ **MERKE**
> Ein Konflikt ist kein Fehler und nichts ist kaputt. Er heißt nur: Git braucht eine Entscheidung von dir.

9.2 Konflikt im Pull Request auf GitHub

Tom und Lara haben beide Ticket #2 bearbeitet. Toms Pull Request ist schon gemergt. In Laras PR steht jetzt im Merge-Kasten: *This branch has conflicts that must be resolved*. Jemand hat dieselbe Stelle geändert und war schneller.

Bei einfachen Textdateien wie `docs/team.md` löst du das direkt im Browser:

1. **Resolve conflicts.** Im Merge-Kasten auf den Knopf klicken. Ein Editor öffnet sich, mit den Markierungen aus Kapitel 3.
2. **Richtige Fassung bauen.** Entscheiden, was gelten soll. Oft ist die Antwort: beide Zeilen behalten, jede in ihrer eigenen Tabellenzeile.
3. **Markierungen löschen.** Alle Zeilen mit `<<<<<<<`, `=======` und `>>>>>>>` entfernen.
4. **Mark as resolved.** Oben rechts klicken, sobald keine Markierung mehr übrig ist.
5. **Commit merge.** GitHub erzeugt den Merge-Commit auf deinem Branch.

Vorher, im Editor

```
<<<<<<< docs/team-lara
| Git- & Prozess-Coach | Lara | @lara-beispiel |
=======
| Git- & Prozess-Coach | Tom | @tom-beispiel |
>>>>>>> main
```

Nachher, aufgelöst

Nach kurzer Absprache bleibt Tom Git-Coach, Lara trägt sich als Design ein:

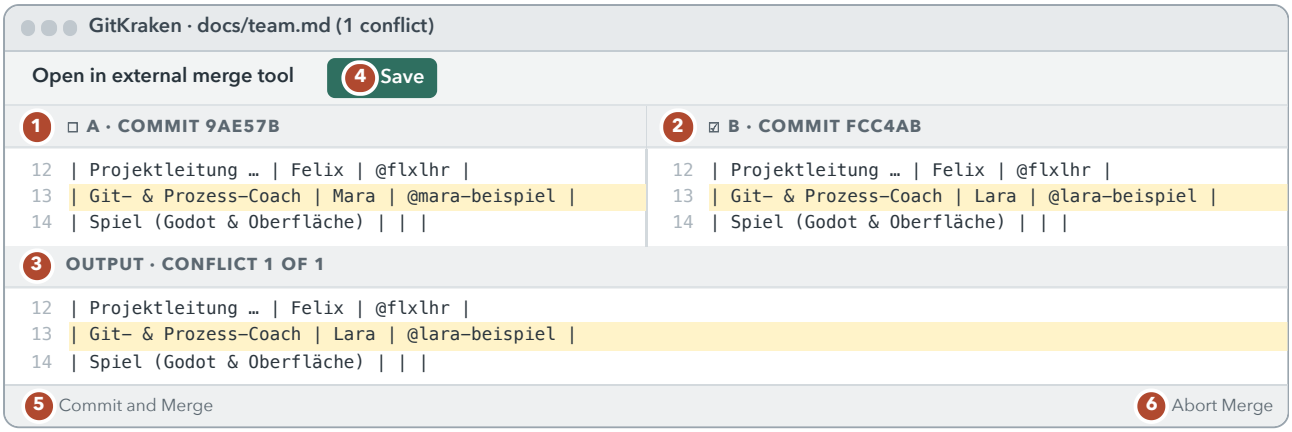
```
| Git- & Prozess-Coach | Tom | @tom-beispiel |
| Design (Grafik & UX) | Lara | @lara-beispiel |
```

Geklappt, wenn: Der Merge-Kasten keinen Konflikt mehr meldet. Jetzt fehlt nur noch das Review.

> ▲ **NICHT JEDER KONFLIKT GEHÖRT IN DEN BROWSER**
>
> Den Web-Editor bietet GitHub nur für einfache Fälle an. Bei Konflikten in vielen Dateien oder in Code, den du testen musst, löst du ihn lieber in GitKraken auf deinem Rechner und startest das Spiel danach einmal.

9.3 Konflikt in GitKraken

In GitKraken begegnet dir ein Konflikt beim Merge oder beim Pull, zum Beispiel bei *Merge main into …*. Oben im Graph erscheint ein Hinweis, dass der Merge angehalten wurde, und rechts steht die betroffene Datei unter den Konflikt-Dateien. Ein Klick darauf öffnet den **Merge-Editor**:



1. **A:** die eine Fassung, mit Commit-Hash. Per Häkchen übernimmst du ihre Zeilen.
2. **B:** die andere Fassung. Hier ist das Häkchen gesetzt.
3. **Output:** das Ergebnis, so wie es gespeichert wird. Hier kannst du auch von Hand tippen.
4. **Save:** speichert das Ergebnis und markiert die Datei als gelöst.
5. **Commit and Merge:** schließt den Merge mit einem Commit ab.
6. **Abort Merge:** bricht ab und stellt den Stand vor dem Merge wieder her.

Abb. 9.1 Der Merge-Editor von GitKraken, nachgebaut. Oben stehen beide Fassungen nebeneinander, unten das Ergebnis.

1. **Datei öffnen.** Rechts die Datei mit Konflikt anklicken.
2. **Auswählen.** Per Häkchen Zeilen aus A, aus B oder aus beiden übernehmen. Im Output siehst du sofort das Ergebnis.
3. **Save.** Die Datei ist jetzt gelöst.
4. **Commit and Merge.** Damit ist der Merge abgeschlossen. Danach pushen.

> ★ **ABORT MERGE IST DER NOTAUSGANG**
>
> Wenn du nicht weiterweißt oder merkst, dass du die andere Fassung nicht verstehst: *Abort Merge*. Alles ist wieder wie vor dem Merge, nichts ist verloren. Dann in Ruhe fragen.

9.4 Wann du nicht selbst auflöst

Ein Konflikt in einer Tabellenzeile ist schnell gelöst. Bei manchen Dateien ist Auflösen von Hand aber gefährlich.

DATEI	WAS TUN
Text, Markdown (`docs/…`)	selbst auflösen, bei inhaltlichen Fragen kurz absprechen
Code (`.gd`, `.php`)	selbst auflösen, danach das Spiel oder die Tests laufen lassen
Godot-Szenen (`.tscn`)	**nicht von Hand mergen.** *Abort Merge* und mit der anderen Person reden
Bilder, Sounds	Git kann sie nicht mischen. Eine Fassung wählen, nach Absprache

▲ EINE SZENE, EINE PERSON

`.tscn`-Dateien sehen aus wie Text, sind aber eine von Godot erzeugte Beschreibung mit IDs und Verweisen. Ein falsch aufgelöster Konflikt darin lässt die Szene nicht mehr laden. Darum arbeitet an einer Szene immer nur eine Person gleichzeitig, und das Board zeigt, wer (→ Kapitel 10, S. 53).

Zusammenfassung

- Main vor jedem Branch pullen, kleine PRs und zwischendurch *Merge main into …* halten Konflikte klein.
- Im Pull Request löst du einfache Konflikte mit *Resolve conflicts*, Markierungen löschen, *Mark as resolved*, *Commit merge*.
- In GitKraken wählst du im Merge-Editor per Häkchen Zeilen aus A und B, speicherst mit *Save* und schließt mit *Commit and Merge* ab.
- *Abort Merge* bringt dich jederzeit zurück zum Stand vor dem Merge.
- Konflikte in `.tscn`-Szenen löst du nicht von Hand. Abbrechen und reden.

? SELBSTTEST

1. Du arbeitest seit einer Woche an deinem Branch, und auf main wurde viel gemergt. Was tust du, bevor du den PR stellst?
2. Im PR steht *This branch has conflicts*. Der Konflikt betrifft nur `docs/team.md`. Wo und wie löst du ihn?
3. Im Merge-Editor setzt du bei A und bei B ein Häkchen. Was steht dann im Output?
4. Du bekommst einen Konflikt in `game/screens/lobby_screen.tscn`. Was tust du?

TEIL IV

Regeln und Erste Hilfe

Wie wir zusammenarbeiten, was tabu ist und wie du dich aus jeder Lage befreist.

10	Regeln und Best Practices	Namen, Ablauf, Godot, Sicherheit
11	Erste Hilfe	Wenn etwas schiefgeht
12	Deine erste Mission – Ticket #2	Alles an einem echten Fall

TEIL IV · REGELN UND ERSTE HILFE

10 Regeln und Best Practices

Regeln sind kein Selbstzweck. Jede einzelne hier verhindert einen Ärger, den Teams immer wieder erleben: Branches, die niemand zuordnen kann, Reviews, die eine Woche liegen bleiben, kaputte Godot-Szenen, Passwörter im öffentlichen Repo. Deshalb steht bei jeder Regel der Grund dabei. Die verbindliche Fassung steht in CONTRIBUTING.md im Repo.

DAS KANNST DU DANACH

- Branch-Namen und Commit-Nachrichten nach unserem Schema schreiben
- den Wochenrhythmus und die Review-Regeln einhalten
- Godot-Dateien richtig behandeln und VS Code richtig öffnen
- die acht Sicherheitsregeln nennen und begründen

HÄNGT ZUSAMMEN MIT

8 Die Runde · 6 Pull Request · 11 Erste Hilfe

Im Video: Kapitel 09 ab 17:48 (Namen 17:48, Ablauf 18:43, Godot und VS Code 19:30, Sicherheit 20:06).

10.1 Gute Namen

Branch-Namen

Ein Branch-Name sagt in drei Teilen, wozu der Branch gehört: Art, Issue-Nummer, Stichwort.

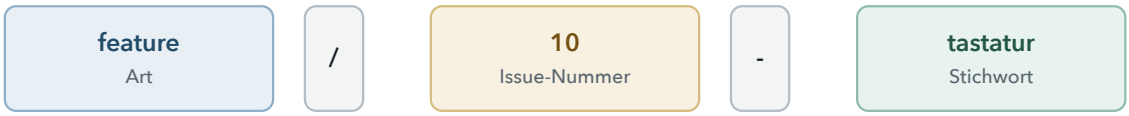


Abb. 10.1 Das Schema für Branch-Namen. Art und Nummer machen den Branch auffindbar, das Stichwort lesbar.

ART	WOFÜR	BEISPIEL
feature/	neue Funktion	`feature/10-tastatur`
fix/	Fehler beheben	`fix/27-host-verlaesst-lobby`
docs/	Doku, Texte	`docs/team-lara`

Alles klein, Bindestriche statt Leerzeichen, keine Umlaute. **Warum:** Wer in drei Wochen die Liste der Branches sieht, soll ohne Nachfragen wissen, was wohin gehört.

So

`feature/10-tastatur`
`fix/27-host-verlaesst-lobby`
`docs/team-lara`

Nicht so

`test` · `mein-branch` · `final2`
`Feature/Tastatur Neu`

Commit-Nachrichten

Deutsch, kurz, und sie sagen, **was sich ändert**. Ein Commit ist eine zusammenhängende Änderung. **Warum:** Die Geschichte soll sich lesen lassen wie ein Protokoll. Bei einem Fehler findet man so schnell den Commit, der ihn eingeführt hat.

So

Tastatur färbt Treffer grün
Fehler beim Beitreten behoben
Wortanzeige trennt Wörter mit Abstand

Nicht so

update · asdf · fix fix fix
Sonntag 23:58 alles

◆ EIN COMMIT, EINE SACHE

Lieber oft und klein committen als ein Riesen-Commit am Sonntagabend. Kleine Commits lassen sich lesen, reviewen und notfalls einzeln zurücknehmen.

10.2 Ablauf, Reviews und Rhythmus

REGEL	WARUM
Vor jedem neuen Branch main pullen.	Sonst arbeitest du auf einem alten Stand und erzeugst unnötige Konflikte.
Pull Requests klein halten. Ein Issue ist eine Sache, die eine Person in etwa einer Woche schafft. Wird es größer: aufteilen.	Kleine PRs werden schneller reviewt und haben weniger Konflikte.
Reviews innerhalb von zwei Tagen, spätestens bis Mittwoch.	Ein liegengebliebener PR blockiert die Person, die ihn gestellt hat.
Freundlich und konkret reviewen. „Was passiert, wenn die Lobby leer ist?“ statt „falsch“.	Eine konkrete Frage kann man beantworten. „Falsch“ nicht.
Nur freigeben, was du verstehst. Im Zweifel im PR nachfragen.	So verstehen mindestens zwei Leute jede Änderung in main.

Unsere Woche



Abb. 10.2 Ein Sprint geht von Donnerstag bis Mittwoch. Donnerstag ist Treffen, Mittwoch sind die Reviews fertig, dazwischen arbeitet jede Person, wann es passt.

Im Treffen machen wir jede Woche dasselbe: **Rückblick** (Was ist fertig, was hängt, wer braucht Hilfe?), **Planung** (Welche Issues kommen in den Sprint, wer nimmt was?) und **Protokoll** (reihum schreibt eine Person mit, in `docs/meetings/`). Absprachen zwischen den Treffen laufen über Discord und in den Issues.

10.3 Godot und VS Code

REGEL	WARUM
Eine Szene, eine Person. An einer `.tscn`-Datei arbeitet immer nur eine Person gleichzeitig. Das Board zeigt, wer woran sitzt.	Szenen lassen sich kaum zusammenführen (→ Kapitel 9, S. 48).
`.uid`**-Dateien mit committen.** Sie liegen neben deinen Skripten.	Godot verweist über diese IDs auf Dateien. Fehlen sie, brechen Verweise bei den anderen.
`.godot/` **und** `build/` **nie committen.** Das regelt `.gitignore`.	Das sind Zwischenspeicher und Bauergebnisse, die jeder Rechner selbst erzeugt.
Vor dem Commit `gdformat game/`**.**	Die CI prüft die Formatierung. So bleibt sie grün (→ Kapitel 6, S. 33).
In VS Code immer den ganzen Repo-Ordner öffnen.	Nur dann zeigt VS Code Git-Status und Einstellungen des Projekts richtig an. Der Ordner `.vscode/` mit deinen Einstellungen bleibt lokal.

★ GODOT ÄNDERT NEBENBEI DATEIEN

Godot schreibt beim Öffnen manchmal in `project.godot` oder in Szenen, die du nur angesehen hast. Vor dem Stagen kurz prüfen, ob nur deine Dateien in der Liste stehen (→ Kapitel 8, S. 42).

10.4 Sicherheit und Verhalten

Unser Repo ist **öffentlich**. Alles, was dort landet, kann die ganze Welt lesen, auch alte Commits, Issues und Kommentare. Das ist gewollt: Das Projekt ist Open Source und zeigt, was wir können. Es heißt aber auch, dass Bots das Repo nach Passwörtern durchsuchen.

▲ WAS EINMAL COMMITTET IST, STEHT IN DER GESCHICHTE

Ein Passwort, das du committest und danach löschst, ist nicht weg. Es steht im alten Commit, und wegen der Hash-Kette lässt sich dieser Commit nicht unauffällig ändern (→ Kapitel 2, S. 13).

GEHÖRT NIE INS REPO	GEHÖRT REIN
Passwörter, Zugangsdaten, `server/config.php`	Code, Doku und Entscheidungen
API-Schlüssel und Tokens	eigene Grafiken und Sounds
private Daten wie Adressen oder Noten	fremde Grafiken, Sounds, Schriften nur mit passender Lizenz

Die acht Regeln

REGEL	WARUM
Kein Force Push. Fragt GitKraken danach: abbrechen.	Force Push überschreibt die Geschichte auf GitHub und damit womöglich die Arbeit anderer.
Nie direkt auf main. Alles läuft über Pull Requests.	main ist geschützt, und jede Änderung braucht Review und grüne Prüfungen (→ Kapitel 6, S. 33).
Keine Zugangsdaten. Nicht im Code, nicht in Issues, nicht auf Screenshots.	Das Repo ist öffentlich, und Gelöschtes bleibt im Verlauf.
Lizenzen beachten. Grafiken, Sounds, Schriften aus dem Netz nur mit passender Lizenz.	Wir veröffentlichen alles. Fremdes ohne Lizenz ist eine Urheberrechtsverletzung.
Keine fremden Add-ons ohne Absprache. Godot-Plugins und Downloads erst im Team klären.	Plugins können Fehler, Sicherheitslücken oder unpassende Lizenzen mitbringen.
Nur freigeben, was du verstehst.	Ein Approve ist eine Unterschrift.
KI-Code prüfen. KI darf helfen, aber wer einen PR stellt, muss jede Zeile erklären können.	Code, den niemand versteht, kann niemand warten oder reparieren.
Zwei-Faktor-Anmeldung an. Unter Settings → Password and authentication, dazu ein eigenes Passwort nur für GitHub.	Wer dein Konto übernimmt, kann in unserem Repo Schaden anrichten.

◆ **FEHLER SIND OKAY. VERSTECKEN NICHT.**

Wenn etwas passiert, sag es. Ein früh gemeldetes Problem ist meist in Minuten gelöst. Ein verstecktes wird groß (→ Kapitel 11, S. 58).

Zusammenfassung

- Branches heißen Art, Schrägstrich, Issue-Nummer, Bindestrich, Stichwort: `feature/10-tastatur`. Alles klein, Bindestriche statt Leerzeichen.
- Commit-Nachrichten sind deutsch, kurz und sagen, was sich ändert. Ein Commit, eine Sache.
- Vor jedem Branch main pullen, PRs klein halten, Reviews in zwei Tagen, freundlich und konkret.
- Ein Sprint geht von Donnerstag bis Mittwoch, Donnerstag ist Treffen.
- Eine Szene, eine Person. `.uid` mit committen, `.godot/` und `build/` nie.
- Das Repo ist öffentlich: keine Zugangsdaten, kein Force Push, nie direkt auf main, Lizenzen beachten, Zwei-Faktor an.

? SELBSTTEST

1. Bilde den Branch-Namen für Issue #31 „Punkte nach jeder Runde anzeigen“.
2. Was ist an der Commit-Nachricht „Änderungen“ falsch? Schlag eine bessere für das Einfärben der Tastatur vor.
3. Ein PR von Tom liegt seit Freitag ohne Review. Es ist Mittwoch. Was sagt unsere Regel?
4. Du findest im Netz ein schönes Hangman-Sprite-Set. Was klärst du, bevor es ins Repo kommt?
5. Eine KI hat dir eine Funktion geschrieben, die funktioniert, die du aber nicht ganz verstehst. Darfst du sie in einen PR packen?

TEIL IV · REGELN UND ERSTE HILFE

11 Erste Hilfe

Es wird etwas schiefgehen. Das ist normal und gehört zum Lernen. Die gute Nachricht: In Git geht fast nie etwas verloren, solange es einmal committet war. Dieses Kapitel ist ein Fehlerkatalog. Du suchst die Lage, in der du steckst, und findest den Ausweg. Vorher ein Grundsatz für alle Fälle: erst fragen, dann klicken.

DAS KANNST DU DANACH

- einen falschen, noch nicht gepushten Commit mit Undo zurücknehmen
- eine Commit-Nachricht mit Amend korrigieren
- Änderungen mit Stash beiseitelegen und mit Pop zurückholen
- einen abgelehnten Push richtig lösen
- im echten Notfall richtig reagieren

HÄNGT ZUSAMMEN MIT

2 Commits · 4 Remote · 7 GitKraken · 10 Regeln

Im Video: Kapitel 10 ab 20:51.

11.1 Erst die Lage, dann der Klick

Bevor du etwas reparierst, beantworte zwei Fragen. Sie entscheiden fast immer, welcher Ausweg der richtige ist.

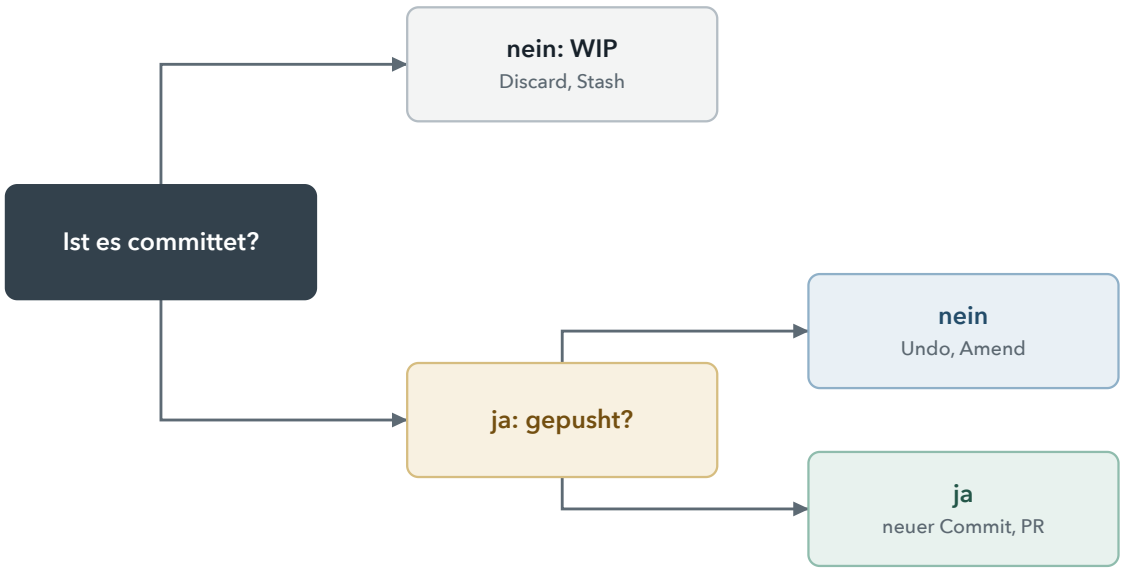


Abb. 11.1 Zwei Fragen sortieren fast jedes Problem. Was nur auf deinem Rechner liegt, kannst du frei reparieren. Was gepusht ist, reparierst du mit einem neuen Commit.

◆ **WAS COMMITTET IST, GEHT FAST NIE VERLOREN**

Jeder Commit bleibt in der Geschichte, selbst wenn kein Branch mehr auf ihn zeigt. Gefährlich sind nur zwei Dinge: verworfene Änderungen, die nie committet waren, und Force Push. Bei allem anderen gibt es einen Weg zurück, im Zweifel mit Hilfe des Git-Coachs.

11.2 Der Fehlerkatalog

Falsch committet, noch nicht gepusht

Lage: Du hast committet, merkst aber, dass etwas fehlt oder falsch ist. Am Branch steht `1↑`.

Ausweg: Oben links auf **Undo**. GitKraken nimmt den letzten Commit zurück. Deine Änderungen bleiben als WIP erhalten, nichts ist verloren. Jetzt korrigieren und neu committen.

Nachricht vertippt oder Datei vergessen

Lage: Der Commit ist fast richtig, aber die Nachricht hat einen Tippfehler, oder eine Datei fehlt.

Ausweg: Die vergessene Datei stagen, beim nächsten Commit das Häkchen **Amend previous commit** setzen, Nachricht korrigieren, committen. Der letzte Commit wird ersetzt statt ein neuer angehängt.

▲ **UNDO UND AMEND NUR VOR DEM PUSH**

Beide schreiben die Geschichte auf deinem Rechner um. Solange nichts gepusht ist, merkt das niemand. Nach dem Push würde dein Rechner nicht mehr zu GitHub passen, und der nächste Push würde abgelehnt. Nach dem Push korrigierst du mit einem neuen Commit.

Aus Versehen auf main committet

Lage: Du hast vergessen, einen Branch anzulegen, und auf `main` committet.

Ausweg: Pushen klappt ohnehin nicht, `main` ist geschützt. Also: Commit mit **Undo** zurücknehmen, die Änderungen bleiben als WIP. Dann oben auf **Branch**, Namen nach Schema eingeben, und dort committen. Ungespeicherte Änderungen wandern beim Anlegen des Branches mit.

Mitten in der Arbeit den Branch wechseln

Lage: Du bist halb fertig, sollst aber kurz einen anderen Branch ansehen, zum Beispiel für ein Review.

Ausweg: **Stash** legt deine ungespeicherten Änderungen in eine Schublade. Dein Arbeitsverzeichnis ist sauber, du kannst wechseln. Zurück auf deinem Branch holt **Pop** sie wieder heraus.

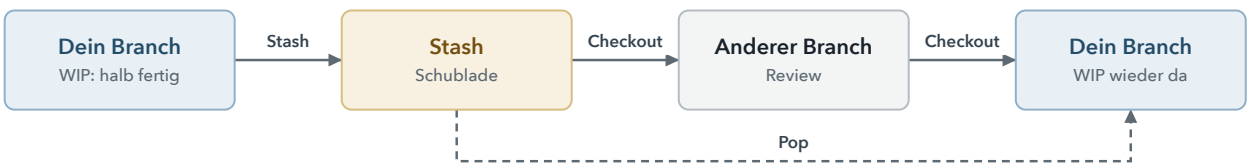


Abb. 11.2 Stash und Pop. Die Schublade hält deine halbfertigen Änderungen, während du woanders bist.

Push abgelehnt: rejected

Lage: GitKraken meldet beim Push, dass er abgelehnt wurde (*rejected*). Am Branch steht `1↑ 1↓`.

Ausweg: Auf GitHub gibt es auf deinem Branch etwas Neues, das du noch nicht hast, zum Beispiel einen Merge-Commit aus *Resolve conflicts*. Erst **Pull**, dann noch einmal **Push**. Niemals Force.

Änderungen verworfen

Lage: Du hast auf **Discard** geklickt und die Änderung war noch nie committet.

Ausweg: Sofort **Undo**. GitKraken kann den letzten Discard zurückholen, solange er deine letzte Aktion war. Hast du danach schon etwas anderes getan, gibt es keinen sicheren Weg mehr, denn einen Papierkorb gibt es nicht. Ist die Datei in deinem Editor noch offen, kann dessen eigenes Rückgängig (Strg / Cmd + Z) helfen. Darum: Discard nur, wenn du die Änderung wirklich nicht mehr willst.

CI rot oder Konflikt im PR

Lage: Ein Check im PR ist rot, oder GitHub meldet einen Konflikt.

Ausweg: Ganz normal. Bei roter CI *Details* lesen, lokal beheben, pushen (→ Kapitel 6, S. 33). Bei Konflikt nach → Kapitel 9, S. 48 vorgehen. Hängst du fest: im PR fragen.

11.3 Der einzige echte Notfall

> ▲ **PASSWORT ODER TOKEN COMMITTET**
>
> **Sofort Felix Bescheid sagen und das Passwort oder den Token ändern.** Erst danach aufräumen. Löschen allein reicht nicht, es steht im Verlauf, und das Repo ist öffentlich. Bots durchsuchen GitHub gezielt nach solchen Funden, oft innerhalb von Minuten.

Warum zuerst ändern? Weil das alte Passwort ab dem Push als bekannt gelten muss. Das Entfernen aus der Geschichte ist aufwendig und schützt nicht vor jemandem, der es schon kopiert hat. Ein neues Passwort dagegen macht das alte sofort wertlos.

11.4 Kurzübersicht

LAGE	AUSWEG	GEHT VERLOREN?
Falsch committet, nicht gepusht	Undo	nein, bleibt als WIP
Nachricht vertippt, Datei vergessen	Amend previous commit, vor dem Push	nein
Auf main committet	Undo, Branch anlegen, dort committen	nein
Branch wechseln mitten in der Arbeit	Stash, später Pop	nein
Push rejected	Pull, dann Push. Nie Force	nein
Discard geklickt	sofort *Undo*, später keiner sicher	ja, sobald du weitergeklickt hast

LAGE	AUSWEG	GEHT VERLOREN?
CI rot	Details lesen, beheben, pushen	nein
Konflikt	auflösen oder Abort Merge	nein
Passwort committet	sofort Felix, Passwort ändern	Passwort gilt als bekannt

Zusammenfassung

- Erst klären: Ist es committet? Ist es gepusht? Danach richtet sich der Ausweg.
- Undo nimmt einen nicht gepushten Commit zurück, Amend korrigiert ihn. Beide nur vor dem Push.
- Stash legt Änderungen beiseite, Pop holt sie zurück.
- Ein abgelehnter Push wird mit Pull und erneutem Push gelöst, nie mit Force.
- Discard lässt sich nur direkt danach mit Undo zurückholen. Danach ist verloren, was nie committet war.
- Ein committetes Passwort ist der einzige echte Notfall: sofort melden und ändern.

? SELBSTTEST

1. Du hast committet und gepusht und siehst dann einen Tippfehler im Code. Undo, Amend oder neuer Commit?
2. Du hast drei Dateien geändert und sollst schnell Toms PR-Branch auschecken, um ihn zu testen. Was machst du mit deinen Änderungen?
3. Warum ist ein Force Push als Lösung für *rejected* gefährlich?
4. Du merkst, dass in deinem letzten gepushten Commit ein API-Schlüssel steht. Was tust du als Erstes, und warum genügt Löschen nicht?

TEIL IV · REGELN UND ERSTE HILFE

12 Deine erste Mission – Ticket #2

Jetzt du. Deine erste Aufgabe im echten Repo ist Ticket #2: Trag deine Rolle in docs/team.md ein. Die Änderung ist winzig, aber du gehst dabei einmal die ganze Runde. Und weil alle dieselbe Datei ändern, wird es Konflikte geben. Das ist Absicht: So übst du sie, solange nichts auf dem Spiel steht.

DAS KANNST DU DANACH

- Ticket #2 vom Assignee bis zum aktuellen main selbst erledigen
- entscheiden, ob dein PR `Closes #2` oder `Teil von #2` schreibt
- einen Konflikt in docs/team.md lösen
- im Übungs-Repo ein erstes Review abgeben

HÄNGT ZUSAMMEN MIT

8 Die Runde · 9 Konflikte · 6 Pull Request

Im Video: Kapitel 11 ab 22:10.

12.1 Die Aufgabe

FELD	WERT
Issue	#2 · Rolle in `docs/team.md` eintragen
Labels	typ: lernen · good first issue
Meilenstein	M0 Setup & Lernen
Fertig, wenn	jede Person steht mit Name und GitHub-Namen in ihrer Zeile, über einen eigenen Branch und Pull Request
Dein Branch	`docs/team-<dein-name>`, zum Beispiel `docs/team-lara`

Ticket #2 ist ein gemeinsames Ticket: Alle fünf arbeiten daran, jede Person mit eigenem Branch und eigenem Pull Request.

12.2 Die Runde für Ticket #2

Eine Checkliste zum Abhaken. Die Einzelheiten zu jedem Schritt stehen in → Kapitel 8, S. 42.

1. **Ticket.** Issue #2 öffnen, *assign yourself*. Den Status setzt bei diesem gemeinsamen Ticket die erste Person auf *In Arbeit*.
2. **Branch.** Im Issue unter *Development* auf *Create a branch*, Name `docs/team-<dein-name>`, Quelle `main`.
3. **Auschecken.** In GitKraken `main` doppelklicken, *Pull*, dann deinen Branch unter *Remote* doppelklicken. Häkchen prüfen.
4. **Ändern.** `docs/team.md` in VS Code öffnen, in deiner Zeile Name und GitHub-Namen eintragen, speichern.
5. **Committen.** Nur `docs/team.md` stagen, Nachricht zum Beispiel `Lara als Git-Coach eingetragen`, *Commit*.
6. **Pushen.** *Push*, beim ersten Mal bestätigen. Laptop und Wolke prüfen.
7. **Pull Request.** *Compare & pull request*, klarer Titel, in die Beschreibung `Teil von #2`. Nur wer als Letztes dran ist, schreibt `Closes #2` (siehe unten).
8. **Review.** Jemand anderes gibt frei. Du selbst reviewst im Gegenzug einen anderen PR zu #2.
9. **Merge.** Nach Freigabe und grünen Prüfungen *Squash and merge*.
10. **main pullen.** `main` auschecken, *Pull*, alten Branch löschen.

12.3 Closes oder Teil von?

Fünf Pull Requests gehören zu einem Ticket. Würde jeder `Closes #2` schreiben, schlösse schon der erste Merge das Ticket, und die Karte stünde auf *Fertig*, obwohl vier Leute noch fehlen.

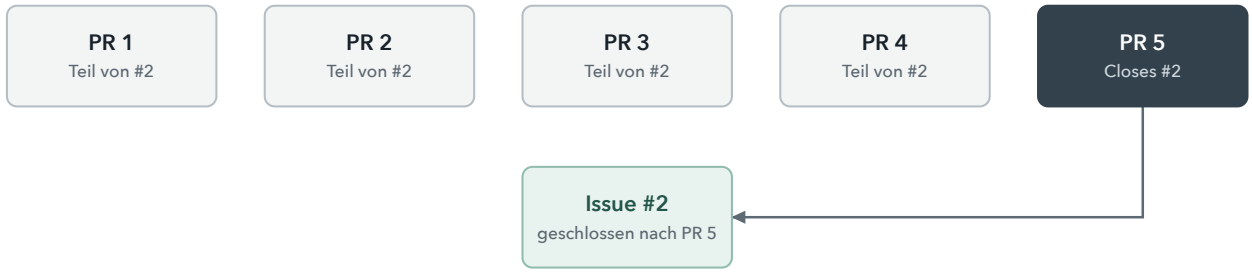


Abb. 12.1 Fünf PRs, ein Ticket. Nur der letzte PR schließt #2, alle anderen verweisen nur darauf.

◆ **MERKE**

Nur der Pull Request, der eine Aufgabe wirklich abschließt, schreibt `Closes #…`. Alle anderen schreiben `Teil von #…`.

Wer ist der letzte? Das klärt ihr kurz im Issue oder auf Discord. Wer merkt, dass alle anderen schon gemergt sind, ergänzt in seinem PR einfach `Closes #2`. Die Beschreibung lässt sich jederzeit bearbeiten.

12.4 Der Konflikt, der kommen wird

Alle ändern Zeilen derselben Tabelle in derselben Datei. Solange zwei Leute verschiedene Zeilen ändern, führt Git das automatisch zusammen. Tragen sich aber zwei in dieselbe Zeile ein, oder ändert jemand versehentlich eine Nachbarzeile mit, meldet GitHub im PR: *This branch has conflicts*.

1. **Nicht erschrecken.** Genau dafür ist die Übung da.
2. **Resolve conflicts** im PR anklicken. Im Editor stehen beide Fassungen zwischen den Markierungen.
3. **Beide Einträge behalten.** Meistens ist die richtige Lösung, dass beide Namen in ihren Zeilen stehen. Gibt es zwei Leute für dieselbe Rolle, kurz absprechen.
4. **Markierungen löschen**, *Mark as resolved*, *Commit merge*.
5. **Lokal nachziehen.** In GitKraken steht an deinem Branch jetzt `1↓`: der Merge-Commit von GitHub. Wenn du noch weiterarbeitest, erst *Pull*.

Ausführlich mit Beispiel: → Kapitel 9, S. 48.

12.5 Üben ohne Risiko: das Übungs-Repo

Neben dem echten Repo gibt es `flxlhr/pangmam-uebung`. Dort darfst du alles ausprobieren: Branches anlegen, Konflikte erzeugen, Undo testen. Nichts davon landet im Spiel.

> • DEIN ERSTES REVIEW
>
> Im Übungs-Repo wartet ein offener Pull Request, **PR #8 „Lara als Git-Coach eingetragen“**, auf dein Review. Öffne *Files changed*, kommentiere eine Zeile über das Plus und schick mit *Review changes* eine Entscheidung ab. So hast du jeden Knopf aus → Kapitel 6, S. 33 einmal gedrückt, bevor es ernst wird.

12.6 Danach

Mit Ticket #2 hast du die Runde einmal ganz gedreht. Die nächsten Aufgaben unterscheiden sich nur im vierten Schritt, dem Ändern. Alles davor und danach bleibt gleich.

Nächste Aufgabe finden

Auf dem Board in *Bereit*, oder mit `label:"good first issue"` filtern.

Hängen geblieben

Kapitel 11 aufschlagen. Dann im PR, auf Discord oder beim Git-Coach fragen.

Merksatz

Vorher pullen, nachher pushen. Und bei Fragen: fragen.

Zusammenfassung

- Ticket #2 ist deine erste Runde: Rolle in `docs/team.md` eintragen, über eigenen Branch `docs/team-<name>` und eigenen Pull Request.
- Alle Teil-PRs schreiben `Teil von #2`, nur der letzte `Closes #2`.
- Konflikte in `team.md` sind gewollt. Beide Einträge behalten, Markierungen löschen, *Mark as resolved*, *Commit merge*.
- Im Übungs-Repo `flxlhr/pangmam-uebung` übst du gefahrlos, PR #8 wartet auf dein Review.
- Die Runde ist bei jeder Aufgabe dieselbe. Nur das Ändern unterscheidet sich.

? SELBSTTEST

1. Du bist die dritte Person, die einen PR zu #2 stellt. Was schreibst du in die Beschreibung?
2. Dein PR meldet einen Konflikt in `docs/team.md`: Mia und du habt beide die Zeile *Server* geändert. Wie gehst du vor?
3. Nach *Commit merge* auf GitHub willst du lokal weiterarbeiten. Was zeigt GitKraken an deinem Branch, und was tust du?
4. Wozu gibt es das Übungs-Repo, wenn Ticket #2 doch schon eine Übung ist?

ANHANG

Nachschlagen

Spickzettel, Glossar, Lösungen und Quellen.

A	**Spickzettel**	Das Meistgebrauchte auf zwei Seiten
B	**Glossar**	Alle Begriffe in einem Satz
C	**Lösungen zu den Selbsttests**	Zum Nachprüfen, nicht zum Vorher-Lesen
D	**Quellen und Weiterlesen**	Wo die Fakten herkommen

ANHANG · NACHSCHLAGEN

A Spickzettel

Das Wichtigste zum Nachschlagen während der Arbeit. Ausdrucken, neben den Bildschirm legen, nach der dritten Runde weglegen.

Die Runde und die Anzeigen

Die Runde	
1 Ticket	GitHub: assign yourself, Status *In Arbeit*
2 Branch	GitHub: *Development → Create a branch*, Quelle `main`
3 Auschecken	GitKraken: `main` · *Pull* · Branch unter *Remote* doppelklicken
4 Ändern	VS Code (ganzer Repo-Ordner) oder Godot, speichern
5 Committen	WIP anklicken · nur deine Dateien stagen · *Commit summary* · *Commit*
6 Pushen	*Push*, beim ersten Mal bestätigen
7 Pull Request	*Compare & pull request* · Titel · `Closes #…` oder `Teil von #…`
8 Review	andere Person: *Files changed* · *Review changes* · *Approve*
9 Merge	*Squash and merge* · *Confirm squash and merge*
10 main pullen	`main` doppelklicken · *Pull* · alten Branch löschen

Was GitKraken anzeigt	
✓ Häkchen	hier bist du (HEAD)
// WIP	Änderungen ohne Commit
1↑	Commit nur bei dir · pushen
1↓	Commit wartet auf GitHub · pullen
1↑ 1↓	erst pullen, dann pushen
Laptop + Wolke	Branch lokal und auf GitHub

Board	
Backlog → Bereit	im Donnerstagstreffen
Bereit → In Arbeit	**du selbst**, plus Assignee
In Arbeit → Review	automatisch, wenn PR verknüpft
Review → Fertig	automatisch beim Merge

Begriffe, Namen, Notfälle

Begriffe: GitKraken · Befehl	
Stage	`git add`
Commit	`git commit`
Push · Pull · Fetch	`git push` · `git pull` · `git fetch`
Checkout (Doppelklick)	`git switch <branch>`
Branch	`git switch -c <branch>`
Stash · Pop	`git stash` · `git stash pop`

Namen	
Branch	`feature/10-tastatur` · `fix/27-host-verlaesst-lobby` · `docs/team-lara`
Commit	deutsch, kurz, was sich ändert: *Tastatur färbt Treffer grün*
Nie	`test` · `final2` · *update* · *asdf*

Wenn etwas schiefgeht	
Falsch committet, nicht gepusht	*Undo*
Nachricht vertippt	*Amend previous commit* (vor dem Push)
Auf main committet	*Undo* · *Branch* · dort committen
Branch wechseln mitten in der Arbeit	*Stash*, später *Pop*
Push rejected	*Pull*, dann *Push*. Nie Force
Konflikt in `.tscn`	*Abort Merge*, mit der Person reden
Passwort committet	sofort Felix · Passwort ändern

Adressen	
Repo	github.com/flxlhr/Pangmam
Board	Reiter *Projects* im Repo
Regeln	`CONTRIBUTING.md`
Wer macht was	`docs/team.md`
Üben	github.com/flxlhr/pangmam-uebung
Treffen	jeden Donnerstag
Merksatz	**Vorher pullen, nachher pushen.**

ANHANG · NACHSCHLAGEN

B Glossar

Jeder Begriff aus Heft und Video in einem Satz, mit dem Kapitel, in dem er erklärt wird.

A bis F

Abort Merge
Bricht einen Merge in GitKraken ab und stellt den Stand davor wieder her. Kap. 9

Amend
Ersetzt den letzten, noch nicht gepushten Commit, etwa um die Nachricht zu korrigieren. Kap. 11

Approve
Freigabe eines Pull Requests im Review. Kap. 6

Arbeitsverzeichnis
Die normalen Dateien im Repo-Ordner, die du öffnest und änderst. Kap. 2

Assignee
Die Person, die gerade an einem Issue arbeitet. Kap. 5

Board
Unser Kanban-Board mit den Spalten Backlog, Bereit, In Arbeit, Review, Fertig. Kap. 5

Branch
Beweglicher Zettel, der auf einen Commit zeigt. Ein eigener Arbeitszweig. Kap. 3

Checkout
Den Branch wechseln. HEAD springt um, die Dateien werden ausgetauscht. Kap. 3

CI (Continuous Integration)
Automatische Prüfungen bei jedem Pull Request, bei uns mit GitHub Actions. Kap. 6

Clone
Das ganze Repo einmal von GitHub herunterladen. Kap. 4

Closes #...
Zeile in der PR-Beschreibung, die das Issue beim Merge schließt. Kap. 6

Commit
Schnappschuss des ganzen Projekts mit Hash, Autor, Datum, Nachricht und Eltern. Kap. 2

Discard
Verwirft eine Änderung im Arbeitsverzeichnis. Nur direkt danach per Undo zurückholbar. Kap. 2, 11

Eltern-Commit
Der Vorgänger eines Commits. Über die Eltern entsteht die Kette. Kap. 2

Fast-Forward
Merge, bei dem nur der Zettel vorrückt, weil main nichts Neues hat. Kap. 3

Fetch
Neue Commits von GitHub holen, ohne sie zu übernehmen. Rückt nur `origin/…` vor. Kap. 4

Force Push
Push, der die Geschichte auf GitHub überschreibt. Bei uns verboten. Kap. 10

G bis W

Git
Das Programm, das Versionen speichert, Branches verwaltet und zusammenführt. Kap. 1

GitHub
Webseite mit der gemeinsamen Kopie, Issues, Board, Pull Requests und CI. Kap. 1

GitKraken
Oberfläche, mit der du Git per Klick bedienst. Kap. 1, 7

.gitignore
Datei, die festlegt, welche Dateien Git nicht beachtet. Kap. 2, 10

Hash
Aus dem Inhalt berechnete Nummer eines Commits, 40 Zeichen lang. Kap. 2

HEAD
Zeiger auf den Branch, auf dem du gerade bist. In GitKraken das Häkchen. Kap. 3

Issue
Ticket mit Nummer für eine Aufgabe. Kap. 5

Konflikt
Rückfrage von Git, wenn zwei Leute dieselbe Stelle unterschiedlich geändert haben. Kap. 3, 9

Label
Etikett zum Sortieren von Issues nach Typ, Priorität und Bereich. Kap. 5

main
Der gemeinsame, geschützte Hauptzweig. Kap. 3, 6

Merge
Zwei Branches zusammenführen. Kap. 3

Meilenstein
Große Etappe bis zur Abgabe, von M0 bis M4. Kap. 5

origin
Name des Remotes auf GitHub. Kap. 4

Pop
Holt mit Stash beiseitegelegte Änderungen zurück. Kap. 11

Pull
Fetch plus Merge: neue Commits holen und übernehmen. Kap. 4

Pull Request (PR)
Anfrage auf GitHub, einen Branch nach main zu übernehmen. Kap. 6

Push
Eigene Commits auf GitHub hochladen. Kap. 4

Remote
Kopie des Repos an einem anderen Ort, bei uns auf GitHub. Kap. 4

Repository (Repo)
Projektordner mit eingebautem Gedächtnis im Ordner `.git`. Kap. 2

Request changes
Review-Entscheidung „bitte noch ändern“. Kap. 6

Review
Eine andere Person liest und prüft deine Änderungen im Pull Request. Kap. 6

Squash and merge
Merge, der alle Commits eines Branches zu einem Commit auf main zusammenpresst. Kap. 6

Stage
Eine Änderung auf den Packtisch legen, damit sie in den nächsten Commit kommt. Kap. 2

Staging-Bereich (Index)
Der Packtisch zwischen Arbeitsverzeichnis und Repository. Kap. 2

Stash
Ungespeicherte Änderungen beiseitelegen. Kap. 11

Undo
Macht in GitKraken die jeweils letzte Aktion rückgängig, etwa einen nicht gepushten Commit oder einen Discard. Kap. 7, 11

WIP (Work in Progress)
In GitKraken die Zeile für Änderungen, die noch in keinem Commit stecken. Kap. 7

ANHANG · NACHSCHLAGEN

C Lösungen zu den Selbsttests

Erst selbst antworten, dann hier nachsehen. Wo deine Antwort anders formuliert ist, aber dasselbe meint, ist sie richtig.

Teil I: Git verstehen

✓ LÖSUNGEN ZU KAPITEL 1 · ÜBERBLICK

1. Ohne Git gewinnt, wer zuletzt speichert. Die Änderung der anderen Person ist still weg. Mit Git committet jede Person auf ihrem Branch, und beim Zusammenführen bleiben beide Änderungen erhalten. Betreffen sie dieselbe Zeile, fragt Git nach, statt still zu überschreiben.
2. GitHub ist eine Webseite, auf der Git-Repositories liegen. Auf dem Rechner arbeitet immer Git, hier über GitKraken. Ohne Git gibt es keine Commits und keine Branches.
3. Nein. Ein Commit liegt nur in deinem lokalen Repo, der linken Hälfte der Landkarte. Erst Push bringt ihn zu GitHub.
4. Branch, Pushen, Pull Request, Merge, main pullen.

✓ LÖSUNGEN ZU KAPITEL 2 · COMMITS

1. Die Dateien im aktuellen Stand bleiben, aber die ganze Geschichte, alle Branches und die Verbindung zu GitHub sind weg. Der Ordner ist kein Repo mehr. Nicht gepushte Commits sind verloren, alles andere lässt sich neu klonen.
2. Nein. Jede noch so kleine Änderung ergibt einen völlig anderen Hash. Ähnliche Inhalte haben keine ähnlichen Hashes.
3. Der geänderte Commit bekommt einen neuen Hash, und damit ändern sich alle Hashes danach. Seine Geschichte passt nicht mehr zu der auf GitHub, der Push wird abgelehnt.
4. WIP anklicken, nur `keyboard.gd` stagen (*Stage File*), Nachricht schreiben, committen. `team.md` bleibt unter *Unstaged Files*.
5. *Unstage* nimmt eine Datei vom Packtisch, die Änderung bleibt im Arbeitsverzeichnis. *Discard* verwirft die Änderung.

✓ LÖSUNGEN ZU KAPITEL 3 · BRANCHES

1. Ein Branch ist eine winzige Datei mit einem Hash. Beim Anlegen kopiert Git keine einzige Projektdatei.
2. Nein. Sie ist auf deinem Branch committet. Auf `main` gibt es sie noch nicht, deshalb verschwindet sie beim Checkout. Beim Zurückwechseln ist sie wieder da.
3. Nein. Git rückt den Zettel `main` per Fast-Forward vor. (Auf GitHub entsteht trotzdem ein neuer Commit, weil wir mit Squash and merge mergen.)
4. Bei Zeile 12 und 14 nicht, Git übernimmt beide Änderungen. Ändern beide Zeile 12 unterschiedlich, gibt es einen Konflikt.
5. Die Zeile `=======` landet als Text in der Datei. In `team.md` steht dann eine kaputte Zeile, in Code wäre es ein Syntaxfehler. Alle Markierungen müssen vor dem Commit raus.

✓ LÖSUNGEN ZU KAPITEL 4 · REMOTE

1. Der Push. Committen ist lokal.
2. Bei Fetch nicht, es rücken nur die Zettel wie `origin/main` vor. Bei Pull ja, dein Branch rückt nach, und die Dateien ändern sich.
3. Zwei Commits liegen nur bei dir. Pushen.
4. Erst `main` pullen. Sonst startet dein Branch auf einem alten Stand, und es drohen unnötige Konflikte.
5. Pull, dann noch einmal Push.

Teil II: GitHub

✓ LÖSUNGEN ZU KAPITEL 5 · ISSUES UND BOARD

1. Das Issue nicht auch noch nehmen. Ein anderes aus *Bereit* wählen, oder im Issue bei Mia nachfragen, ob sie Hilfe braucht.
2. Alle Punkte der Liste „Fertig, wenn“ sind erfüllt, und das Issue ist geschlossen.
3. Eine zweite Person hält die Aufgabe für frei und fängt dieselbe an. Ihr baut doppelt, und bei Godot-Szenen drohen kaum lösbare Konflikte.
4. `good first issue`. Unter *Issues* ins Suchfeld `label:"good first issue"` eingeben.

✓ LÖSUNGEN ZU KAPITEL 6 · PULL REQUEST

1. Den Pull Request gibt es in Git nicht. Er ist eine Funktion von GitHub und wird im Browser gestellt.
2. `Teil von #10`. Nur der PR, der das Issue abschließt, schreibt `Closes #10`.
3. Auf *Details* klicken, Meldung lesen, lokal beheben (oft reicht `gdformat game/`), committen, pushen. Die CI läuft dann neu.
4. Nein. Du committest und pushst auf denselben Branch, der PR aktualisiert sich von selbst.
5. Weil main geschützt ist. Deine Änderungen kommen nur über einen PR mit Review und grünen Prüfungen dorthin. Dein Branch betrifft bis dahin nur dich.

Teil III: Die Praxis in GitKraken

✓ LÖSUNGEN ZU KAPITEL 7 · GITKRAKEN

1. In der rechten Leiste, nachdem du die WIP-Zeile angeklickt hast: *Staged Files*.
2. Der Branch existiert nur auf deinem Rechner. Er wurde noch nie gepusht. Push fehlt.
3. Am Häkchen neben dem Branch-Namen in der linken Leiste.
4. Nein. WIP heißt, die Änderung steckt noch nicht einmal in einem Commit.

✓ LÖSUNGEN ZU KAPITEL 8 · DIE RUNDE

1. GitKraken hat den neuen Branch noch nicht gesehen. `main` auschecken und *Pull* (oder auf den automatischen Fetch warten). Danach steht er unter *Remote → origin*.
2. `project.godot` nicht mit committen. Unstagen und, wenn du sicher bist, dass die Änderung ungewollt ist, verwerfen. Im Zweifel fragen.
3. Der Push hat nicht geklappt, zum Beispiel weil der Dialog beim ersten Push nicht bestätigt wurde. Meldung lesen und noch einmal pushen.
4. `main` auschecken und pullen. Den alten Branch lokal löschen.
5. Weil seine Commits per Squash zu einem Commit auf main geworden sind. Der alte Branch passt nicht mehr zur Geschichte von main. Jede Aufgabe bekommt einen frischen Branch von einem aktuellen main.

✓ LÖSUNGEN ZU KAPITEL 9 · KONFLIKTE

1. `main` pullen, dann Rechtsklick auf `main` und *Merge main into …*. Eventuelle Konflikte bei dir lösen, das Spiel testen, pushen.
2. Direkt im PR auf GitHub: *Resolve conflicts*, richtige Fassung bauen, Markierungen löschen, *Mark as resolved*, *Commit merge*.
3. Beide Zeilen, die aus A und die aus B, untereinander.
4. *Abort Merge* und mit der anderen Person reden. Szenen werden nicht von Hand gemergt.

Teil IV: Regeln und Erste Hilfe

✓ LÖSUNGEN ZU KAPITEL 10 · REGELN

1. Zum Beispiel `feature/31-punkte-anzeigen`.
2. „Änderungen“ sagt nicht, was sich ändert. Besser: `Tastatur färbt Treffer grün`.
3. Reviews sollen innerhalb von zwei Tagen fertig sein, spätestens bis Mittwoch. Heute ist die Frist, also jetzt reviewen oder im PR klären, wer es übernimmt.
4. Die Lizenz: Dürfen wir die Grafiken verwenden, verändern und öffentlich weitergeben, und muss der Urheber genannt werden? Dann im Team absprechen.
5. Nein, nicht so. Wer einen PR stellt, muss jede Zeile erklären können. Erst verstehen, dann einreichen.

✓ LÖSUNGEN ZU KAPITEL 11 · ERSTE HILFE

1. Ein neuer Commit, der den Fehler behebt. Undo und Amend nur vor dem Push.
2. *Stash*, Toms Branch auschecken und testen, zurückwechseln, *Pop*.
3. Er überschreibt die Geschichte auf GitHub. Commits, die andere gepusht haben, können dabei verschwinden.
4. Sofort Felix Bescheid sagen und den Schlüssel ändern beziehungsweise sperren. Löschen genügt nicht, weil er im alten Commit steht und das Repo öffentlich ist.

✓ LÖSUNGEN ZU KAPITEL 12 · ERSTE MISSION

1. `Teil von #2`, es sei denn, alle anderen sind schon gemergt. Dann `Closes #2`.
2. Mit Mia klären, wer *Server* übernimmt. Dann *Resolve conflicts*, die richtigen Einträge in ihre Zeilen, Markierungen löschen, *Mark as resolved*, *Commit merge*.
3. `1↓` am Branch, das ist der Merge-Commit von GitHub. Vor dem Weiterarbeiten pullen.
4. Im Übungs-Repo kannst du Riskantes gefahrlos ausprobieren: Konflikte erzeugen, Undo testen, Reviews üben. Nichts davon landet im Spiel.

ANHANG · NACHSCHLAGEN

D Quellen und Weiterlesen

Die Regeln und Abläufe stammen aus der Kickoff-Präsentation des Teams und aus CONTRIBUTING.md. Die Fakten zu Git, GitHub und GitKraken sind an der offiziellen Dokumentation geprüft, Stand November 2026. Die Bezeichnungen in GitKraken entsprechen Version 12.6.

Projekt

- Repository: github.com/flxlhr/Pangmam, darin `CONTRIBUTING.md`, `README.md` und `docs/team.md`
- Übungs-Repository: github.com/flxlhr/pangmam-uebung
- Kickoff-Präsentation des Teams, November 2026
- Erklärvideo v2 „Git, GitHub & GitKraken“, 23 Minuten, mit Kapitelmarken

Dokumentation, nach Kapiteln

Kapitel 1 · Überblick

Pro Git: Was ist Versionsverwaltung?
https://git-scm.com/book/de/v2/Erste-Schritte-Was-ist-Versionsverwaltung%3F

Pro Git: Was ist Git?
https://git-scm.com/book/de/v2/Erste-Schritte-Was-ist-Git%3F

Kapitel 2 · Commits

Pro Git: Änderungen nachverfolgen und im Repository speichern
https://git-scm.com/book/de/v2/Git-Grundlagen-%C3%84nderungen-nachverfolgen-und-im-Repository-speichern

Kapitel 3 · Branches

Pro Git: Branches auf einen Blick
https://git-scm.com/book/de/v2/Git-Branching-Branches-auf-einen-Blick

Pro Git: Einfaches Branching und Merging
https://git-scm.com/book/de/v2/Git-Branching-Einfaches-Branching-und-Merging

Kapitel 4 · Remote

Pro Git: Mit Remotes arbeiten
https://git-scm.com/book/de/v2/Git-Grundlagen-Mit-Remotes-arbeiten

Pro Git: Remote-Branches
https://git-scm.com/book/de/v2/Git-Branching-Remote-Branches

GitKraken Desktop: Push, Pull und Fetch
https://help.gitkraken.com/gitkraken-desktop/pushing-and-pulling/

Kapitel 5 · Issues und Board

GitHub Docs: About issues
https://docs.github.com/en/issues/tracking-your-work-with-issues/about-issues

GitHub Docs: About Projects
https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

Kapitel 6 · Pull Request

GitHub Docs: About pull requests
https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

GitHub Docs: About pull request reviews
https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

GitHub Docs: Linking a pull request to an issue
https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue

GitHub Docs: About protected branches
https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

GitHub Docs: About merge methods on GitHub
https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/about-merge-methods-on-github

GitHub Docs: Understanding GitHub Actions
https://docs.github.com/en/actions/about-github-actions/understanding-github-actions

gdtoolkit (gdformat, gdlint)
https://github.com/Scony/godot-gdscript-toolkit

Kapitel 7 · GitKraken

GitKraken Desktop Help
https://help.gitkraken.com/gitkraken-desktop/

GitKraken Desktop: Interface
https://help.gitkraken.com/gitkraken-desktop/interface/

GitKraken Desktop: GitHub-Integration
https://help.gitkraken.com/gitkraken-desktop/github-gitkraken-desktop/

Kapitel 9 · Konflikte

GitHub Docs: Resolving a merge conflict on GitHub
https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github

GitKraken Desktop: Branching, Merging und Konflikte
https://help.gitkraken.com/gitkraken-desktop/branching-and-merging/

Kapitel 10 · Regeln

Godot Docs: Version control systems
https://docs.godotengine.org/en/stable/tutorials/best_practices/version_control_systems.html

Kapitel 11 · Erste Hilfe

GitKraken Desktop: Undo und Redo
https://help.gitkraken.com/gitkraken-desktop/undo-and-redo/

GitKraken Desktop: Stashing
https://help.gitkraken.com/gitkraken-desktop/stashing/